6809 Forth — Open Items Checklist

Everything below was explicitly flagged during the build as incomplete, unverified, or deliberately deferred. Nothing here is a surprise — each item was named at the point it came up. This is a consolidated list to work from, not a new set of findings. Regenerated to reflect fixes and design decisions made since the original version.

Known bugs — resolved since the original checklist

- **`DUMP` 's partial-final-line bug** — fixed via `DUVALID` tracking (`25_tools_word_set.asm`). The ASCII column no longer reads past the valid bytes on a short final line.
- **`SUBSTITUTE`** — completed. It now performs a real bounds-checked copy (prefix / replacement / suffix) via a shared `SUBCOPY` helper, reusing `SEARCHW` for the match. Still scoped to a single registered name/value pair, not a full table — see "Open design questions" below; that scope limit is a deliberate simplification, not a bug.
- **`VALUE` 's PFA** — moved from `CODEHERE` (immutable space) to `VARHERE` (mutable space), matching `VARIABLE` 's pattern, so every `TO` -driven update now writes into the correct region instead of into code space. Not on the original checklist (found and fixed after this list was first generated); logged here for the record.
- **Every scratch/global cell now has a real `RMB` -assigned address in page zero** (`00_memory_map_and_globals.asm`), applied rather than left as a placeholder. Two real findings came out of doing this: the budget is **253 of 256 bytes used — only 3 bytes of headroom** in the GLOBALS page, and `SNEND` (documented in the original placeholder list) turned out to be genuinely dead — never read or written anywhere — and was dropped rather than given an address. Any future scratch cell added to this system will need to fit in that remaining 3 bytes or the page-zero fast-addressing property (DP = `$00`) stops covering it.
- **The ROM base dictionary is now real** (`27_forth_dictionary.asm`) — every primitive with actual code got a real header, chained via `LINK` , `CFA` pointing straight at its code label. `ABORTHDR` 's `LINK` field, a placeholder `0` since it was first built, is now resolved to the chain's newest entry. Building this surfaced two real findings, not just mechanical work: the original 1024-byte `BASEDICT` could not hold the header table (ultimately 1954 bytes needed, once `DOES>` was added — see below), so it was resized to 2048 bytes, taking the space from `BASECODE` (now 6080 bytes, was 7104) — a resize based purely

on the header-table budget, not on any actual measurement of `BASECODE`'s assembled size, which has never been checked with a real 6809 assembler. Generating this table also surfaced that **`DOES>` had no corresponding code anywhere in the file** — resolved in a follow-up pass, see below.

- **`DOES>` — resolved.** `SETDOES` (the patching runtime) was added beside `DODOES` / `DOESRT0`, and `DOES>` itself (code label `DOESGT`, since a literal `">"` is not a valid 6809 assembler label) was added right after `CREATE`. `DOESBEH` was added to the GLOBALS layout to support it, using 2 of the page's last 3 free bytes — only 1 byte of headroom now remains in page zero. `DOES>` also has a real dictionary header now (`H_DOESGT`, the chain's newest entry); `ABORTHDR`'s `LINK` was updated again to point at it.
- **Four internal loop-label names were reused across unrelated routines: `FLOOP`, `UDLOOP`, `DRPOS`, `CMLOOP`.** Resolved — each pair renamed to a distinct name scoped to its own routine: `FIND`'s `FLOOP` → `FFLOOP`; `FILLW`'s `FLOOP` → `FILLOOP`; `UDIV16`'s `UDLOOP` → `UD16LOOP`; `UDDIGIT`'s `UDLOOP` → `UDDLOOP`; `DIVCOMMON`'s `DRPOS` → `DCRPOS`; `DDOTR`'s `DRPOS` → `DDRPOS`; `CMOVEW`'s `CMLOOP` → `CMVLOOP`; `COMPAREW`'s `CMLOOP` → `CMPLOOP`. Verified zero duplicate labels and zero stray references to any of the old names remain anywhere in the file.
- **Hardware (RTS) serial handshaking — implemented.** `IRQH`'s receive path and `KEY` now toggle RTS based on the input ring's fill level (`INFILL`, `RTSCHECKHI`, `RTSCHECKLO` against `INHIWATER`=48/`INLOWATER`=16, with hysteresis between them) via a new `CR_RTSHI` control byte and `RTSSTATE` flag — the last free byte in the GLOBALS page, which is now fully packed at 256/256. CTS (the other handshaking direction) needed no firmware logic at all: confirmed against the 6850 datasheet that TDRE is automatically inhibited while CTS is deasserted, so this system's existing TDRE-gated transmit logic already respects it. A real race was identified and closed: mainline code (`KEY`, via `RTSCHECKLO`) and the ISR (`IRQH`'s `TXOFF`) can both decide to write the control register, so `RTSCHECKLO` masks IRQ around its critical section and `TXOFF` checks `RTSSTATE` before writing. Software (XON/XOFF) handshaking remains unimplemented — see below.
- **`TRUE` and `FALSE` — implemented.** Surfaced while organizing the ANS test suite: neither existed as an actual dictionary word, only as the internal `TRUEV` / `FALSEV` assembler constants. Added as `CONSTANT TRUE -1` / `CONSTANT FALSE 0` (`TRUEBODY` / `FALSEBODY`, section 26; headers `H_TRUE` / `H_FALSE`, chain's two newest entries, section 27) — the first `CONSTANT`-pattern ROM-resident words in this system. Every other ROM word's CFA is a plain code label; a `CONSTANT`'s CFA is the `DODOES`-trampoline pattern instead, built here with fixed, assemble-time addresses rather than a `CODEHERE` snapshot, since there is no interactive `CREATE` / `CONSTANT` phase for ROM content.

- **One genuine 6309-only instruction (`TSTD`) was in use — fixed.** Found by auditing every mnemonic in the file against the real 6809 instruction set, rather than trusting the Build Instructions section's earlier claim that no 6309 extensions were used (that claim was wrong until this fix). `TRYNUM` (section 9) used `TSTD` with no operand to test whether `D` was zero after `PULU D` — `PULU` / `PULS` don't affect condition codes on genuine 6809 hardware, unlike a load, so this relied on a 6309-only convenience instruction. Replaced with `CMPD #0` , a real 6809 instruction with identical behavior for this purpose. A full mnemonic audit after the fix found zero remaining non-6809 instructions anywhere in the file.

- **`BASELATEST` was an undefined symbol — a real assembly-breaking bug, not a placeholder.** `COLD` (section 4) has referenced `LDD #BASELATEST` to initialize `LATEST` since this file's earliest version, and the SECTION 27 header comment has long asserted " `BASELATEST` remains `QUITHDR` " - but no `EQU` or label actually named `BASELATEST` existed anywhere in the file. This would have failed to assemble outright. Fixed by adding `BASELATEST EQU QUITHDR` directly after `QUITHDR` 's own definition (section 26), matching what the comment always said it should equal. Verified: exactly one definition, no duplicate symbols, and `COLD` 's forward reference to it resolves under standard two-pass assembly.

- **`JSR CR` (bare, undefined) appeared at five call sites — a real assembly-breaking bug, not a naming inconsistency.** The actual routine has always been labeled `CRW` (section 22), following this file's own convention of giving short/symbolic ANS names a distinct code label. `ABORT` , `QUIT` 's error-report path, `QUIT` 's `ok` -prompt path, `WORDS` (`WWDONE`), and `DUMP` (`DULEND`) all called the bare, undefined `CR` instead. Fixed at each site to `JSR CRW` , preserving each call site's original spacing exactly. Verified: `CRW` defined exactly once, referenced 7 times total, zero remaining bare- `CR` instruction operands anywhere in the file (the two harmless bare "CR" occurrences that remain are a section-title comment and the dictionary header's `FCC "CR"` name string, neither of which is a bug).

- **Two ALU instructions used invalid register-to-register syntax (`ADDA B` , `EORA B`) — the 6809 has no such addressing mode, so the assembler read `B` as an undefined direct-page symbol.** Found in the single-cell multiply routine (`ADDA B` , twice) and `DOPLUSTEST` 's sign comparison for `+LOOP` (`EORA B`). Fixed with the standard 6809 idiom for combining two registers: `PSHS B` followed by `ADDA ,S+` / `EORA ,S+` (push B, then operate through the auto-incrementing stack-indexed operand). Verified: a full sweep of every ALU instruction (`ADDA` / `ADDB` / `SUBA` / `SUBB` / `ANDA` / `ANDB` / `ORA` / `ORB` / `EORA` / `EORB` / `CMPA` / `CMPB` / `ADCA` / `ADCB` / `SBCA` / `SBCB` / `BITA` / `BITB`) against a bare `A` / `B` operand found zero remaining instances; every other bare `B` in the file (in `STA` / `LDA` / `LEAX ...,X` and `PSHS B`) is genuine, valid 6809 accumulator-offset

indexed addressing or register-list syntax.

- **Nine scratch symbols (`MRESULT` , `MVCNT` , `MVDST` , `MVSRC` , `FILLCHR` , `FILLCNT` , `FILLADDR` , `HSLEN` , `HSADDR`) were used throughout `MOVE` / `CMOVE` / `CMOVE>` , `FILL` , `HOLDS` , and the single-cell multiply routine but never declared anywhere — a real, assembly-breaking gap.** Auditing every one of the 142 already-declared GLOBALS cells for actual use found zero dead cells to reclaim this time (unlike `SNEND` earlier), and the GLOBALS page was independently confirmed at exactly 256/256 bytes with no headroom. Since `MOVE` -family, `FILL` , `HOLDS` , and the multiply routine never call each other or run concurrently in this single-threaded interpreter, they now share physical storage: three new cells (`MVCNT` , `MVDST` , `MVSRC`) hold the real storage, and `FILLCNT` / `HSLEN` alias `MVCNT` , `FILLADDR` / `HSADDR` alias `MVDST` , and `MRESULT` / `FILLCHR` alias `MVSRC` , all via `EQU` . These three new cells could not fit in the full GLOBALS page, so they live at `$0100` , carved from the front of `USER0` (shrunk from 128 to 122 bytes, now starting at `$0106`) — a real, documented tradeoff: these three cells use ordinary extended addressing, not direct-page, including inside `CMOVEW` 's per-byte copy loop.

- **`JSR SPACE` (bare, undefined) — same class of bug as `CR` , fixed the same way.** `WORDS` called the bare, undefined `SPACE` instead of the actual routine label `SPACEW` (section 22). Fixed to `JSR SPACEW` . Checked `SPACES` for the same pattern (none found — `SPACESW` correctly calls `EMIT` directly) and swept the file for any other bare `SPACE` instruction reference (none remain; the two harmless bare "SPACE" occurrences left are a section-title comment and the dictionary header's `FCC "SPACE"` name string).

- **Three short branches (`BHS` , `BEQ` , `BNE`) overflowed the 8-bit short-branch range (±127 bytes) — a real assembler error, not a style issue.** `SUBCOPY` 's overflow check (`BHS SUBOVERFLOW` , ~87 source lines to target), `DUMPW` 's per-line loop test (`BEQ DUDONE` , ~76 lines), and `DUMPW` 's loop-back (`BNE DULINE` , ~76 lines) all spanned too much code for an 8-bit displacement. Fixed by converting each to its long-branch equivalent (`LBHS` / `LBEQ` / `LBNE`), which uses a 16-bit displacement (±32767 bytes) - functionally identical, just not range-limited.

- **Ten more short branches have a large (41-51 source line) span to their target and were not individually confirmed safe.** Found by a heuristic sweep (line-count as a rough proxy for byte distance, since no real assembler is available in this environment to get an exact count) after fixing the three confirmed overflows above, all of which spanned 76+ lines - these ten are meaningfully shorter but not verified within range: `QUERY` 's `QLOOP` (three branches, lines 579/582/589), `FIND` 's `NOTFOUND` / `FFLOOP` (lines 1210/1256), `ACCEPT` 's `ALOOP` / `ADONE` (lines 1501/1462), `UNESCAPE` 's `UEDONE` / `UELOOP` (lines 3888/3931), and `EMPTY` (line 1153). None of these were reported as failing and none have been changed; if a real assembler flags any of them, the fix is the same:

convert to the matching `LBxx` long-branch form.

- **`ORG BASECODE` was missing entirely — a fundamental placement bug, not a cosmetic gap.** `VECTORS` ($FFF0) and `INIT` ($FFC0) both had their own `ORG`, but `SECTION 3` (`IRQH`) — and every routine through `SECTION 26`, i.e. nearly the entire interpreter — had no `ORG` placing it in `BASECODE` at all. Without it, this code would have continued growing from wherever `INIT`'s WARM message left the location counter, inside `INIT`'s own 48-byte `$FFC0-$FFEF` budget, overflowing directly into `VECTORS` rather than landing in `BASECODE` ($E800-$FFBF) anywhere. Fixed by adding `ORG BASECODE` immediately before `SECTION 3` begins.
- **Superseded by a later, more precise count** (see the "Follow-up" entry below with the 7984-byte instruction-by- instruction result) - the 8660-byte figure here used a cruder heuristic and a 6080-byte budget that's since changed (`BASECODE` is now 8110 bytes nominal, after the `BASECODE` / `BASEDICT` 30-byte shift). Kept as history, not deleted, since it was a genuine finding at the time - just not the current best estimate. Original text: **Adding the missing ORG makes `BASECODE`'s byte budget concretely checkable for the first time — and a rough estimate suggests it may not fit.** This was previously flagged only as "unverified" (no real assembler has ever been run against this file); with a real `ORG` boundary now in place, a heuristic per-instruction byte count (inherent/direct-page/ indexed/immediate/extended opcode sizes, correctly distinguishing direct-page GLOBALS operands from extended ones) over every instruction from `SECTION 3` through `SECTION 26` estimated roughly 8660 bytes against a 6080-byte budget — about 2580 bytes over. The underlying question (does `BASECODE` actually fit, confirmed by a real assembler) is still genuinely open - see the later, more precise follow-up entry.
- **`USER0` and `USER1` (250 bytes of unallocated reserve space, never read or written by any code) were deleted, and the region from `MVSCRATCH` through `APPDICT` was made fully contiguous.** `SERBUF`, `INBUF`, `OUTBUF`, `TIBBUF`, `WORDBUF`, and `SIBUF` all shifted down to sit immediately after `MVSCRATCH` ($0106) with no gaps between any of them. This also closed a separate, pre-existing 11-byte gap between `WORDBUF` and `SIBUF` ($02F5-$02FF) that had nothing to do with `USER0` / `USER1` but was caught while verifying the region was genuinely contiguous end to end. `APPDICT` was extended downward to close the resulting gap too (new start $021B, was $0320) — its end ($6EFF) is unchanged, so this is a pure 261-byte gain in application dictionary space, not a resize of its budget. This was an interpretation of "contiguous," not something explicitly asked for beyond the 6 buffers themselves; flagged here in case a fixed `APPDICT` start was actually intended instead. Verified: zero duplicate symbols, zero remaining references to `USER0` / `USER1` anywhere in the code, and the full `GLOBALS` -> `MVSCRATCH` ->buffers-> `APPDICT` span checked programmatically for zero gaps.

- **APPVARS and APPDICT swapped positions - APPVARS now sits below APPDICT .** APPDICT moved from $021B to $031B (still ending at $6FFF , directly below APPCODE); APPVARS moved from $6F00 down to $021B (same 256-byte size, now sitting directly above the buffers instead of directly below APPCODE). Checked before making the change: only two code references exist for either symbol (COLD initializing VARHERE / DPHERE to each region's start) and neither depends on their relative order, so the swap was safe. Verified: zero duplicate symbols, the whole region from GLOBALS through APPCODE 's start remains contiguous with zero gaps, dictionary chain unaffected (219 entries, since this change doesn't touch it at all).

- **APPVARS grown from 256 to 8000 bytes, taking the space directly from APPDICT .** APPVARS now spans $021B-$215A (start address unchanged); APPDICT shrank by the same 7744 bytes to $215B-$6FFF (end unchanged, still directly below APPCODE), giving 20133 bytes of application dictionary space, down from 27877. Checked before making the change: still only two code references to either symbol (COLD 's VARHERE / DPHERE initialization), neither size-dependent. Verified: zero duplicate symbols, APPVARS size is exactly 8000 bytes, the whole region stays contiguous end to end, dictionary chain unaffected.

- **ACIA (the 256-byte memory-mapped I/O block) renamed to INOUT , and the actual 6850 ACIA chip's registers moved to INOUT+8 instead of the block's base.** The block still spans the same 256 bytes ($DF00-$DFFF); it's now modeled as a general I/O region with the ACIA chip occupying one small part of it (ACIACR / ACIASR = INOUT+8 = $DF08 , and the data register ACIADR at $DF09), leaving INOUT+0 .. INOUT+7 free for other memory-mapped devices sharing the block. Doing this rename surfaced a real, previously-hidden bug: COLDSTRT 's ACIA reset sequence used STA ACIA (the block's base) instead of STA ACIACR (the control register) in two places - this only ever worked because ACIA and ACIACR happened to be the same address in the old scheme. Once ACIACR moved to INOUT+8 , writing to the bare block base would have silently stopped resetting the ACIA at all. Fixed both to STA ACIACR . Verified: INOUT defined once, the bare ACIA symbol (at the time) no longer existed anywhere, and every other ACIA register access in the file already used the correctly-named register symbols rather than the bare block name.

- **ACIA reintroduced as an explicit base symbol (ACIA EQU INOUT+8), with ACIACR / ACIASR now defined relative to it (EQU ACIA) rather than directly to INOUT+8 .** The data register was briefly renamed to ACIRDR in the same pass, then reverted back to ACIADR immediately afterward once flagged as a typo - both code sites (IRQH 's receive and transmit paths) were updated each time the name changed. Verified: each of ACIA / ACIACR / ACIASR / ACIADR defined exactly once, resolving to the

expected values (`ACIA = INOUT+8` , `ACIACR` / `ACIASR = ACIA` , `ACIADR = ACIA+1`), zero remaining references to `ACIRDR` anywhere, zero duplicate symbols, dictionary chain unaffected.

- **`INITEND` / `INITSIZE` landmark constants added at the true end of the INIT block** (`INITEND EQU *` , `INITSIZE EQU *-COLDSTRT`), right after `WARMMSGL` . The request referenced a `COLDSTART` label, which doesn't exist in this file - used the actual existing label `COLDSTRT` instead of introducing a new undefined symbol.

- **CORRECTED: the manual 78-byte count below was wrong - a real assembler run reports `INITCODE` 's actual size as 71 bytes (`$47`).** This entry's own last line said to trust `INITSIZE` once the file was actually assembled rather than this manual count - that's now happened, and the manual count was off by 7 bytes. See the later entry recording the correction in full; kept here as history, not rewritten. Original text: **`INITEND` 's comment now states the invariant explicitly ("value should match vector ORG") - and checking it confirms it currently holds.** The precise instruction-by-instruction byte count from when the `VECTORS` collision was first found (78 bytes exactly, `COLDSTRT` through `WARMMSGL`) gives `INITCODE start ($FFA2) + 78 = $FFF0` , exactly matching `VECTORS ' ORG` . Zero gap, zero overlap - unlike the still-open `INITCODE` / `BASECODE` overlap at the other end of this same region, which remains a real problem. Same caveat as always: this is a manual count, not a real assembler's output, so `INITSIZE` (computed automatically at assembly time) is the figure to trust once this file is actually assembled.

- **`BASECODESTRT` / `BASECODEEND` / `BASECODESIZE` and `BASEDICTSTRT` / `BASEDICTEND` / `BASEDICTSIZE` landmark constants added, matching the `INITEND` / `INITSIZE` pattern - and checking the two stated invariants gives one clean confirmation and one confirmation of an already-known problem.** `BASEDICTEND` (`$D85D` + the exact 1973-byte dictionary content = `$E012`) matches `ORG BASECODE` (`$E012`) precisely - this boundary is genuinely correct, not just close. `BASECODEEND` , using the same rough heuristic estimate flagged much earlier (~8660 bytes, never confirmed with a real assembler) starting from `BASECODESTRT` (`$E012`), lands around `$101E6` - not only failing to match `ORG INITCODE` (`$FFA2`) as the comment expects, but overflowing past `$FFFF` entirely on that estimate. The exact amount of room actually available between `BASECODESTRT` and `INITCODE` ($FFA2 - $E012 = 8080 bytes) is itself less than the ~8660-byte estimate, meaning even filling every available byte up to `INITCODE` with zero gap would still likely fall short. This is the same underlying problem already tracked elsewhere (`BASECODE` possibly not fitting its budget, and separately the `INITCODE` / `BASECODE` overlap), now independently reachable via these new landmarks once the file is actually assembled, rather than a new, different issue.

- **Follow-up: a real instruction-by-instruction count (not the rough heuristic above) puts `BASECODE` 's actual size at 7984 bytes (`$1F30`), 96 bytes (1.2%) short of a target assembler- listing value of `$1F90` (8080 bytes) given for comparison.** Built a mnemonic-and-addressing-mode-aware counter distinguishing inherent/immediate/direct/extended/indexed forms, correct prefix requirements (`$10` / `$11` for `LDY` / `STY` / `LDS` / `STS` / `CMPD` / `CMPY` / `CMPU` / `CMPS`), and true direct-page symbols (only `$0000` - `$00FF` , matching `DP`) - a meaningfully more rigorous pass than the original ~8660-byte estimate. Every one of the 3751 instructions in the region was classified (zero "unrecognized" remaining after fixing early misses like `LSLB` / `LSLA` as `ASLB` / `ASLA` synonyms). Checked for likely sources of the remaining 96-byte gap before accepting it: zero indexed operands use an offset outside the 5-bit range that would need extra encoding bytes (all 125 numeric-offset operands found are small, e.g. `1,X` / `-1,Y`), and the two apparent "indirect `[ ]` " matches were a false positive (a section-title comment, not real indirect addressing). The target value `$1F90` is notable in its own right: `BASECODE + $1F90 = $FFA2` exactly, meaning if the real assembled size actually matched it, `BASECODE` and `INITCODE` would be perfectly contiguous with zero gap and zero overlap - which would also resolve the separate, still-open `INITCODE` / `BASECODE` overlap noted elsewhere. My count doesn't confirm that, though it's close (1.2% off). This is still not a real assembler's output - the standing caveat throughout this file - and the gap could come from encoding edge cases a manual model can approximate but not guarantee against.
- **`ROMSTRT` / `ROMEND` added as the outer ROM boundary (`$C100` / `VECTORS-1`), and `INITCODE` / `BASECODE` / `BASEDICT` verified contained within it - all three pass.** `ROMEND` was initially defined as `$10000` ("one past `VECTORS` 's end," a symbolic value that doesn't fit as a real 16-bit address), then corrected to `VECTORS-1` (`$FFEF` , one before `VECTORS` 's start) - a genuine 16-bit address usable directly in comparisons or as a memory operand, not just symbolic arithmetic. Re-checked after the correction rather than assumed still valid: `INITCODE` ($FFA2-$FFEF), `BASECODE` ($E012-$FFBF), and `BASEDICT` ($D85D-$E011) are all still genuinely within `ROMSTRT..ROMEND` - `INITCODE` 's own end now lands exactly on the new boundary, a tighter and more meaningful check than before, not a violation. This containment check passes cleanly, independent of the other open problems below.
- **`ROMSTRT` / `ROMEND` renamed to `USROMSTRT` / `USROMEND` , each with a short "Usable ROM start"/"Usable ROM end" comment.** Cosmetic, not functional - neither symbol was referenced by any code, only by nearby comments (three sites, all updated: the pair's own definitions and `INOUT` 's cross-reference). Verified: zero remaining bare `ROMSTRT` / `ROMEND` references anywhere, zero duplicate symbols, dictionary chain unaffected.

- **`VECTOREND` / `VECTORSIZE` landmark constants added at the true end of the vector table, matching the `INITEND` / `INITSIZE` pattern - and checking the stated invariant confirms it exactly, not approximately.** Unlike `INITEND` / `BASECODEEND` (which needed a manual, approximate instruction-by-instruction byte count since real 6809 instructions have variable-length encoding), the vector table is pure `FDB` data with a fixed, unambiguous 2-byte width per entry - counting the 8 entries (`VRESV` through `VRESET`) gives exactly 16 bytes, matching the comment's stated `$10` expectation precisely. Verified: zero duplicate symbols, dictionary chain unaffected.

- **`BASECODESTRT` removed as requested - but `BASECODESIZE` depended on it, so removing it alone would have left a genuine dangling undefined-symbol reference, the same bug class found and fixed several times earlier in this file.** Confirmed first that `BASECODESTRT` was numerically identical to `BASECODE` itself (only comments sit between `ORG BASECODE` and where `BASECODESTRT` was defined, zero emitted bytes), then fixed `BASECODESIZE` to read `BASECODEEND-BASECODE` directly instead - matching the same pattern just applied to `INITSIZE` / `INITCODE` , not a new approach invented for this case. Verified: zero remaining `BASECODESTRT` references anywhere, `BASECODESIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

- **`BASEDICTSTRT` removed the same way, for the same reason - `BASEDICTSIZE` depended on it too.** Same fix pattern as `BASECODESTRT` immediately above, applied to its counterpart: confirmed `BASEDICTSTRT` was numerically identical to `BASEDICT` (only `ORG BASEDICT` sits before it, zero emitted bytes), removed it, and repointed `BASEDICTSIZE` to `BASEDICTEND-BASEDICT` directly. Verified: zero remaining `BASEDICTSTRT` references anywhere, `BASEDICTSIZE` resolves cleanly, zero duplicate symbols, dictionary chain unaffected.

- **`INOUT` moved from `$DF00` to `$C000` (with `INOUTEND EQU INOUT+$FF` added immediately after), and every ACIA-related address verified between the two - also passes.** `ACIA` / `ACIACR` / `ACIASR` (`$C008`) and `ACIADR` (`$C009`) all fall within `INOUT` - `INOUTEND` (`$C000` - `$C0FF`), checked numerically. Confirmed there is no other memory-mapped I/O device anywhere in this file besides the ACIA family - nothing else needed checking. This move has a real, positive side effect worth naming clearly: it resolves the INOUT portion of the three-way collision flagged when `BASEDICT` moved to `$D85D` two turns ago (`INOUT` no longer overlaps `BASEDICT` , since `$C0FF < $D85D`). The `DSTACK` and `RSTACK` portions of that same collision were untouched by this specific change, but have since been resolved separately - see below.

- **`INOUT` 's new collision with `APPCODE` is resolved, and so is the rest of the original `BASEDICT` collision - `RSTACK` , `DSTACK` , `CODETOP` , `APPCODE` , and `APPDICT` all moved down `$2000` .** `RSTACK` ($BC00-$BEFF) and `DSTACK` ($B800-$BBFF) no longer overlap

BASEDICT at all - the three-way collision first found when BASEDICT moved to $D85D is now fully resolved (INOUT resolved it two turns ago, this resolves the remaining two thirds). APPCODE 's move ($5000-$B7FF) also separately resolves its overlap with INOUT from the previous entry, as a side effect of the same shift, not a second fix. Re-verified with a full pairwise sweep after the move, not assumed: the only overlap remaining from the four found last turn is the original, unrelated INITCODE / BASECODE one (30 B) - INITCODE / BASECODE / BASEDICT / RSTACK / DSTACK / INOUT / APPCODE are all now mutually clean.

- **APPDICT 's collision with APPVARS is now resolved - APPVARSEND changed from a static APPVARS+8000 to APPDICT-1 , making it self-derive from wherever APPDICT actually sits instead of a fixed size.** APPVARSEND is now $1FFF (was $215B), giving APPVARS 7653 bytes of actual usable space (down from the originally-intended 8000 - the 347-byte reduction exactly matches the overlap this closes). APPVARS 's own EQU still starts at $021B and its comment still cites the historical "8000 bytes" intent, now explicitly marked as the original intent rather than the current usable size. Functionally, APPVARS 's enforced range ($021B-$1FFF) no longer reaches APPDICT ($2000-$6EA4) at all - checked numerically, not assumed. This also makes the boundary self-correcting: if APPDICT moves again, APPVARSEND moves with it automatically, rather than needing another manual fix like the last several turns' worth of address changes required. VUNUSEDW 's own code is unchanged - it already computed against APPVARSEND (see below), so this fix took effect purely by changing what that symbol means. Verified: zero duplicate symbols, dictionary chain unaffected.

- **DSTACK moved up $200 (to $BCFF), resolving both the CODETOP / DSTACK mismatch and the RSTACK / DSTACK gap from last turn in one move.** DSTACK 's new occupied bottom ($B900 , from its $BCFF top and 1024-byte size) now matches CODETOP ($B900) exactly - APPCODE 's nominal ceiling no longer overstates safe growth room, and the 512-byte overlap that created between APPCODE 's nominal range and DSTACK 's true range is gone. As a bonus, not something separately requested: DSTACK 's new top ($BCFF) is now exactly contiguous with RSTACK 's bottom ($BD00), closing the 512-byte gap that had opened between them too. Verified with a full pairwise sweep after the move: exactly two overlaps remain in the current memory map - the still-open APPDICT / APPVARS one (347 B) and the original, unrelated INITCODE / BASECODE overlap (30 B) - down from three last turn.

- **The VECTORS collision found by verifying INITEND is now resolved - INIT 's ORG moved from $FFC0 to $FFA0 .** Widened INIT from 48 to 80 bytes ($FFA0-$FFEF), comfortably covering the ~78 bytes COLDSTRT + WARM + WARMMSG actually needs (2 bytes of margin) - still an estimate from manual byte-counting, not a real assembler's output. INIT 's own ORG was also converted from a literal $FFC0 to symbolic ORG INIT ,

matching `BASECODE` / `BASEDICT` 's convention, so the `EQU` and the actual placement can no longer drift apart the way `BASECODE` 's did before that was caught. One real, unresolved interaction worth naming: `INIT` 's new start ($FFA0) is 32 bytes below the old documented `BASECODE` end ($FFBF), tightening the still-open `BASECODE` overflow risk below by that much further - not a new problem in kind, since that risk was already estimated at roughly 2580 bytes over budget, dwarfing this additional 32-byte reduction, but worth tracking alongside it rather than treated as independent.

- **RESOLVED (since this was written): the `INITCODE` / `BASECODE` overlap described below (30 bytes at the time) was fully closed several turns later** by shifting `BASECODE` and `BASEDICT` both down exactly 30 bytes - `BASECODE` 's nominal end now lands exactly one byte below `INITCODE` 's start, zero gap, zero overlap, confirmed by a full pairwise sweep of the entire memory map. Kept as history below, not deleted. Original text: **`INIT` moved again, from `$FFA0` to `$FFA2` (request contained a `&FFA2` typo - used the correct `$` hex prefix instead).** `INIT` is now exactly 78 bytes ($FFA2-$FFEF), matching the `COLDSTRT` + `WARM` + `WARMMSG` byte-count estimate with zero margin (was 80 bytes, with 2 bytes of slack). This is a genuinely tighter fit than before, worth naming as a real tradeoff: any inaccuracy in the manual byte count, or any future addition to `COLDSTRT` / `WARM` , now has zero room to absorb. The overlap with `BASECODE` noted above is **not resolved by this change, only reduced** - from 32 bytes to 30 bytes ($FFA2-$FFBF). This was a real, pre-existing problem before this turn touched anything (`$FFA0` already overlapped `BASECODE` 's declared end by 32 bytes), not something newly introduced. Not fixed here, in either direction (shrinking `BASECODE` further or moving `INIT` above it instead of below): both are bigger architectural decisions than "change one EQU," and a real assembler run would settle the actual `BASECODE` byte count this depends on.

- **`INIT` renamed to `INITCODE`** (`EQU` and its `ORG` reference, both in the memory-map constants). The section-title comment ("SECTION 2: INIT CODE") and one historical narrative comment describing a past bug scenario (with the byte figures true at that time, not today) were deliberately left referring to "INIT" as plain English/history, not the symbol - renaming a symbol doesn't obligate rewriting prose that correctly describes what was true before the rename existed. Doing this rename surfaced that the documentation's "ROM Size Required" section (Section 2) had gone stale independent of the rename itself: it still asserted the four ROM regions were contiguous at exactly 8192 bytes, which stopped being true as soon as `INIT` moved to `$FFA2` last turn and started overlapping `BASECODE` . Rewritten to state the overlap plainly instead of the now-false claim. Verified: `INITCODE` defined exactly once, zero duplicate symbols, dictionary chain unaffected.

- **`BASEDICT` and `BASECODE` addresses applied exactly as requested, and the**

documented ROM requirement widened to a 16K part - but this surfaced a severe, unresolved collision, not a minor tradeoff. `BASEDICT` moved from `$E000` to `$D85D` - verified before making the change that `$E012 - $D85D = 1973` bytes exactly matches SECTION 27's real, measured dictionary content, a genuine zero-padding exact fit (was 2048 bytes, 75 bytes of slack). `BASECODE` moved from `$E800` down to `$E012` to match; its own upper bound was not given and stayed at `$FFBF`, growing it from 6080 to 8110 bytes.

- **RESOLVED (since this was written): the three-way collision described below is fully closed.** `INOUT` moved first (resolving the `INOUT` third), then `RSTACK` / `DSTACK` / `CODETOP` / `APPCODE` / `APPDICT` were all given new addresses (resolving `DSTACK` / `RSTACK`), and `BASEDICT` itself later moved again to `$D83F` (from `$D85D`, shifting down 30 bytes alongside `BASECODE`) as part of closing the separate `INITCODE` / `BASECODE` overlap. A full pairwise sweep of the current memory map (`VECTORS` / `INITCODE` / `BASECODE` / `BASEDICT` / `INOUT` / `RSTACK` / `DSTACK` / `APPCODE` / `APPDICT` / `APPVARS` /all buffer regions/ `GLOBALS`) confirms **zero overlaps anywhere**, re-checked as part of this same update. Kept as history below, not deleted. Original text: **`BASEDICT` 's new range ($D85D-$E011) entirely overlaps three other live regions: `DSTACK` (931 of its 1024 bytes, `$D85D-$DBFF`), `RSTACK` (all 768 bytes, `$DC00-$DEFF`), and `INOUT` (all 256 bytes, `$DF00-$DFFF`).** Found by checking the new address against every other region's boundaries before updating the documentation - not caught until then. This is a fundamental, system-breaking conflict, not a byte-budget tightness issue like the `INITCODE` / `BASECODE` overlap noted elsewhere: as configured, the ROM dictionary, the data stack, the return stack, and the ACIA's registers would all be mapped to the same physical addresses simultaneously, which cannot work on real hardware without bank switching (which this system does not have). The requested `EQU` values were applied exactly as given, since that's what was asked; resolving the collision itself was not attempted here, since it requires a decision this response can't make alone - moving `DSTACK` / `RSTACK` / `INOUT` elsewhere, choosing a smaller `BASEDICT` / `BASECODE` split that doesn't reach down this far, or reconsidering whether `$D85D` was the address actually intended. The documented ROM part was still widened from 8K to 16K per the request (real total usage of `BASEDICT` + `BASECODE` + `INITCODE` + `VECTORS` alone, ignoring the collision, is about 10.15K, leaving roughly 6.2K of spare capacity within a 16K×8 EPROM), but that figure is secondary to the collision above. Verified: zero duplicate symbols, dictionary chain unaffected (219 entries, since this doesn't touch it).
- **The `INITCODE` / `BASECODE` overlap - open since it was first found, mentioned in at least five separate entries above across several turns - is finally resolved.** `BASECODE` and `BASEDICT` both shifted down exactly 30 bytes (`BASECODE`: `$E012` ->

`$DFF4`; `BASEDICT`: `$D85D` -> `$D83F`), chosen precisely: 30 bytes is exactly the overlap amount, so `BASECODE`'s nominal end (`$FFA1`, unchanged 8110-byte budget) now lands exactly one byte below `INITCODE`'s start (`$FFA2`) - zero gap, zero overlap. `BASEDICT` shifted the same amount to stay perfectly contiguous with `BASECODE`'s new start, preserving its own exact, zero-padding fit. Verified with a full pairwise sweep across every region in the memory map, not just the two that moved: **zero overlaps anywhere** - the first time that's been true since this whole sequence of address changes began. `USROMSTRT` / `USROMEND` containment re-checked and still passes for all three ROM regions. Also cleaned up the accumulated resolved-issue narrative in the top-of-file note (previously ~40 lines chronicling the `BASEDICT` / `DSTACK` / `RSTACK` / `INOUT` / `CODETOP` / `APPDICT` saga turn by turn, including one claim - the `APPDICT` / `APPVARS` overlap being open - that was already stale before this edit) down to a short, current-state summary, matching the same cleanup already applied to the documentation.

- **`forth6809.asm`'s four ORG'd blocks physically reordered to match ascending memory address, low to high: `BASEDICT`, `BASECODE`, `INITCODE`, `VECTORS`** (previously `VECTORS`, `INITCODE`, `BASECODE`, `BASEDICT` - roughly the reverse). Pure text reordering only - `ORG` directives set the actual assembled address independent of where in the file each block appears, so this has zero effect on the assembled output; verified rather than assumed by comparing the sorted multiset of every line in the file before and after, which matched exactly (nothing added, removed, or altered - only position changed). This was also done in a freshly-reset environment (the working directory from earlier turns was gone; recovered by copying the last-delivered `forth6809.asm` back from the persistent output location and confirming its hash matched what was last verified). The dictionary chain-walk check initially appeared to fail after reordering (1 of 219 entries reached) - traced to a bug in the verification script itself, not the file: an arbitrary 300-character cap on how much of each header's body text it scanned cut off `ABORTHDR`'s second `FDB` field, since its comment runs several lines longer than most headers. Fixed the script to scan each header's full body instead of a fixed character count, re-ran, and confirmed all 219 entries reachable end to end. Also confirmed zero duplicate symbols and all four region `EQU`s still resolve to their correct, unchanged addresses.

- **`ROMSTRT` / `ROM:` / `FILL` padding block's 256-byte gap is now closed - the `ORG` target changed from `ROMSTRT` to `USROMSTRT`, and `ROMSTRT` itself moved to the top of the memory-map constants, alongside `USROMSTRT` / `USROMEND` rather than sitting alone just before its point of use.** The gap flagged last turn wasn't a formula problem - `BASEDICT- USROMSTRT` (5951 bytes) was already the right byte count, it just needed to start from `USROMSTRT` ($C100), not `ROMSTRT` ($C000). With `ORG USROMSTRT`, the fill now covers `$C100 - $D83E` exactly, landing with zero gap against `BASEDICT`'s real start (`$D83F`). `ROMSTRT` remains a distinct, real symbol (`$C000`, the true physical EPROM

start, still meaningfully different from `USROMSTRT`), now serving purely as a documented reference constant rather than an `ORG` target. `FILL`'s status as an unconfirmed LWASM directive remains open - that wasn't addressed this turn - see the note left in place at the actual `FILL` line. Also fixed a second stale comment found while making these changes: "BASECODE is $E800" (on the `ORG BASECODE` line, left over from several address changes ago) corrected to $DFF4; "BASEDICT is $E000" was checked and found already correct from a prior turn, needing no change. Verified: zero duplicate symbols, `ROMSTRT` still defined exactly once at its new location, zero remaining `ORG ROMSTRT` anywhere, dictionary chain still 219 entries intact. Also worth noting: this turn's work began by discovering the local working copy had silently diverged from the actually-delivered file (showing `ORG USROMSTRT` when the delivered file actually had `ORG ROMSTRT`) - resolved by re-syncing the working copy from the persistent, hash-confirmed output file before making any changes, rather than editing from an unverified state.

- **A second `FILL` padding block added, this time between `BASECODE`'s real content and `INITCODE`'s start - same intent and same open verification question as the `ROM:` block two turns ago.** `BASEND:` sits right where `BASECODE`'s actual assembled content ends (the same point `BASECODEEND` marks, immediately before `SECTION 2` begins, now that the file's physical section order has `BASECODE` directly followed by `INITCODE`); `FILL $FF,INITCODE-BASEND` covers whatever gap exists between there and `INITCODE`'s start. Genuinely self-correcting, not a hardcoded guess: because `BASEND` is computed from wherever real content actually ends rather than an estimated figure, this fill's size will automatically be correct once a real assembler runs, regardless of the precision of any manual byte count. Using the precise instruction-by- instruction count from several turns ago (recomputed fresh here to confirm it's still 7984 bytes, unchanged since nothing in the intervening turns touched `BASECODE`'s actual content), this would cover 126 bytes (`$FF24` - `$FFA1`) - matching the slack between real content and the nominal 8110-byte budget exactly, as expected. `FILL`'s status as an LWASM directive remains unconfirmed (see the `ROM:` block's own note) - not re-verified here, since the question is identical for both uses. Verified: zero duplicate symbols, `BASEND` defined exactly once, dictionary chain still 219 entries intact.
- **Both padding blocks repositioned, and the `ROM:` block's explanatory comment deleted entirely.** `ORG USROMSTRT` / `ROM:` / `FILL` moved from just before `ORG BASEDICT` to just before `SECTION 27`'s comment header instead (now sitting directly after `GLOBALS_USED`); `BASEND:` / `FILL` moved from just before `ORG INITCODE` to just before `SECTION 2`'s comment header (directly after `BASECODESIZE`). The multi-line "UNVERIFIED: FILL is not a directive..." comment on the `ROM:` block is gone - the underlying uncertainty (whether `FILL` is a real, supported LWASM directive) is

unchanged and still real, it's just no longer written down at that specific spot; see the historical entries above for the actual finding. Pure repositioning and deletion, nothing recomputed or changed in substance. Verified: zero duplicate symbols, `ROM:` / `BASEND:` each still defined exactly once, both blocks confirmed (by text position, not assumption) to sit before their respective section's comment header rather than after, zero remaining "UNVERIFIED" text anywhere, dictionary chain still 219 entries intact.

- **`MVSCRATCH` (the `ORG $0100` block, `MVCNT` / `MVDST` / `MVSRC` and their aliases through `FILLCHR`) moved from right after `GLOBALS EQU $0000` to right after `GLOBALS_USED EQU 256` instead - continuing the same file-order-matches-memory-order cleanup as the `VECTORS` / `INITCODE` / `BASECODE` / `BASEDICT` reordering several turns back.** Moved as one unit: the full explanatory comment block plus all nine lines of actual code, ending exactly at `FILLCHR EQU MVSRC` as specified - `SP0` / `RP0` (unrelated stack- pointer constants that happened to sit right after `MVSCRATCH`) stay in their original position, now following `GLOBALS` directly. Verified with the strongest available check: the sorted multiset of every line in the file, compared against the version delivered immediately before this edit, matched exactly - confirming nothing was added, removed, or altered, only repositioned. Also confirmed the new order is correct (`GLOBALS_USED` -> `MVSCRATCH`'s `ORG $0100` -> `ORG USROMSTRT`, matching ascending memory address: `$0000` - `$00FF` globals, `$0100` - `$0105` MVSCRATCH, `$C100` + ROM), zero duplicate symbols, every `MVSCRATCH`-related constant still defined exactly once, dictionary chain still 219 entries intact.

- **Conditional assembly added for serial I/O: interrupt-driven (original) and polling (new) implementations now coexist, switched by a single flag, `SERIALPOLL EQU 1` (polling is the default/active mode).** Verified LWASM's actual conditional- assembly directives against the real manual before using them (`IFEQ` / `IF` / `IFNE` / `IFDEF` / `IFNDEF` / `ELSE` / `ENDC` are documented) rather than guessing, unlike the still-unresolved `FILL` question elsewhere. Three conditional blocks, all gated by the same flag: (1) `INFILL` / `RTSCHECKHI` / `RTSCHECKLO` / `IRQH` (the full interrupt-driven receive/transmit/RTS-handshaking logic) - only assembled when `SERIALPOLL=0`; when `SERIALPOLL=1`, `IRQH` becomes a bare `RTI` stub, matching the other genuinely unused vectors (`NMIH` / `FIRQH` / `SWI2H` / `SWI3H`), since ACIA interrupts are never enabled in polling mode and the vector table unconditionally references `IRQH` by name either way. (2) `KEY` / `KEYQ` / `EMIT` - both versions compile to the same three label names, so the dictionary headers (`H_KEY` / `H_KEYQ` / `H_EMIT`) never need to change regardless of which mode is active. The new polling versions block (`KEY`, `EMIT`) or check once (`KEYQ`) directly against `ACIASR`'s `SR_RDRF` / `SR_TDRE` bits - no ring buffers, no RTS/CTS, fully synchronous. (3) `COLDSTRT`'s ACIA mode-select, now choosing between `CR_RXON` (RX interrupt enabled) and a new `CR_POLL` (`$15` - `CR_RXON` with only the RX-interrupt-

enable bit cleared; RTS held permanently low, since polling has no ring buffer to overflow and needs no flow control). Checked before writing any of this that no other code references `RTSSTATE` / `INFILL` / `RTSCHECKHI` / `RTSCHECKLO` / `CR_RTSHI` / `CR_RXTX` outside what's already being conditionally wrapped - confirmed clean, nothing left dangling. Verification required a different approach than the usual duplicate-symbol check, since `KEY` / `KEYQ` / `EMIT` / `IRQH` legitimately appear twice in the raw source now (once per branch) - a naive check would false-positive on that. Instead, simulated what LWASM would actually assemble for each value of `SERIALPOLL` (stripping whichever branch is inactive) and ran the real structural checks against each simulated result separately: both branches independently show zero duplicate symbols, `KEY` / `KEYQ` / `EMIT` / `IRQH` each resolving to exactly one definition, and the dictionary chain fully intact (219 entries) in both cases. Also confirmed the three `IFEQ` / `ELSE` / `ENDC` blocks are balanced (3/3/3) in the raw file.

- **SUPERSEDED by a later entry below: moving `ABORTHDR` / `QUITHDR` into `BASEDICT` reintroduced a 19-byte `BASECODE` / `BASEDICT` overlap after this was written** - "zero overlaps anywhere" was accurate at the time, not any more. Kept as history, not deleted. Original text: **Current memory map state, re-verified fresh as of this update: zero overlaps anywhere.** A full pairwise sweep across all 18 regions (`VECTORS` , `INITCODE` , `BASECODE` , `BASEDICT` , `INOUT` , `RSTACK` , `DSTACK` , `APPCODE` , `APPDICT` , `APPVARS` , `SIBUF` , `WORDBUF` , `TIBBUF` , `OUTBUF` , `INBUF` , `SERBUF` idx, `MVSCRATCH` , `GLOBALS`) found no collisions - every gap/overlap that was ever open (the `INITCODE` / `BASECODE` 30-byte overlap, the `BASEDICT` / `DSTACK` / `RSTACK` / `INOUT` three-way collision, the `CODETOP` / `DSTACK` mismatch, the `APPDICT` / `APPVARS` overlap) has been resolved by earlier turns and stays resolved now, after the `SERIALPOLL` conditional-assembly addition and both `FILL` padding blocks, neither of which touch region boundaries. All three ROM-resident regions (`INITCODE` / `BASECODE` / `BASEDICT`) remain genuinely contained within `USROMSTRT..USROMEND` . What remains genuinely open, not a gap/overlap question: whether `BASECODE` 's real assembled size actually fits its 8110-byte nominal budget (the precise manual count is 7984 bytes, still not confirmed by a real assembler - see the earlier follow-up entry), and `FILL` 's status as an actual LWASM directive.

- **`ABORTHDR` / `QUITHDR` / `BASELATEST` moved from their own separate section into `BASEDICT` proper, at the end of the dictionary entries (right after `DICTTOP`), and renamed to `H_ABORT` / `H_QUIT` to match this file's `H_` naming convention.** This wasn't purely cosmetic: these two headers were previously sitting physically inside `BASECODE` 's address range (Section 26 sits before `ORG BASEDICT` in the file), even though they're header *data*, not code - meaning their bytes were being counted against `BASECODE` 's budget instead of `BASEDICT` 's. Moving them into the actual `ORG BASEDICT` ...

`BASEDICTEND` block fixes that miscounting, at the cost of a new, real consequence: `BASEDICT` 's real size grew from 1973 to 1992 bytes (the 19 bytes `H_ABORT` and `H_QUIT` actually occupy), but `BASECODE` 's start (`$DFF4`) is a fixed address that doesn't move just because `BASEDICT` 's real content grew - reintroducing a genuine 19-byte overlap between them (`$DFF4-$E006`), the same class of problem as the original `INITCODE` / `BASECODE` overlap resolved several turns ago. A full pairwise sweep of the entire memory map confirms this is the *only* new overlap - nothing else is affected. Not fixed here, since it needs a decision (shift `BASECODE` by 19 bytes to match, the same kind of fix used last time; or something else) rather than a mechanical follow-up. `TRUEBODY` / `FALSEBODY` (real code, correctly retitled) stayed in `BASECODE` where they belong - only the two header structures moved. Verified: zero duplicate symbols (checked against both `SERIALPOLL` branches via the same simulation-based method established for that feature), dictionary chain still 219 entries, now correctly walked starting from `H_QUIT` rather than `QUITHDR` , ending cleanly at `0` in both configurations. Also fixed two separately-stale items caught while making this edit: the Section 27 header comment still cited `BASEDICT` as `$D85D-$E011` (from before the 30-byte shift several turns back) and the top-of-file summary's "zero overlaps anywhere" claim, both now updated to reflect this change and its real consequence.

- **`BASECODE` shifted up 19 bytes (`$DFF4 -> $E007`) to close the `BASECODE` / `BASEDICT` overlap from last turn - but the overlap wasn't eliminated, it was relocated.** `BASECODE` 's new start does land exactly one byte above `BASEDICT` 's real end (`$E006`), closing that specific 19-byte overlap precisely. But `BASECODE` 's nominal size (8110 bytes) didn't change, so its nominal end shifted by the same 19 bytes too - from `$FFA1` to `$FFB4` , which is now 19 bytes *into* `INITCODE` 's start (`$FFA2`). A full pairwise sweep of the entire memory map confirms exactly one overlap remains, same total byte count (19), just at a different boundary (`INITCODE` / `BASECODE` instead of `BASECODE` / `BASEDICT`). This was checked and reported precisely, not assumed away or glossed over: shifting only `BASECODE` 's *start* while its budget stays fixed cannot reduce total overlap when that budget was already calibrated (in an earlier turn) to exactly reach `INITCODE` 's start from the *old* position - moving the start without shrinking the budget just pushes the same problem to the other end. Actually eliminating both overlaps at once would need `BASECODE` 's nominal size reduced by 19 bytes (to 8091), not just its start address moved; not done here, since that's a different change than what was asked. Verified: zero duplicate symbols in both `SERIALPOLL` branches (same simulation method as before), dictionary chain still 219 entries in both configurations. Also fixed a real arithmetic error caught before delivery: an initial draft of this comment miscalculated the new `INITCODE` overlap as 7 bytes at `$FFAE` - corrected to the actual 19 bytes at `$FFB4` before this was ever shipped.

- **`DICTTOP` confirmed genuinely unused and removed, along with the one comment that named it.** Checked precisely before touching anything: defined once (`DICTTOP EQU H_FALSE`), and referenced exactly one other place in the whole file - inside a prose comment describing the dictionary chain, not by any real code. No `EQU` depended on it, nothing `JSR` 'd or `FDB` 'd it. Confirms the suspicion that raised this: `BASELATEST` (`EQU H_QUIT` , the true overall chain head used by `COLD` to initialize `LATEST`) is what's actually used at runtime; `DICTTOP` was a separate, distinct value (`H_FALSE` , the newest of the 217 generation-pass entries specifically, not the true head) that nothing ever consumed. Removed the `EQU` line and rewrote the one comment that named it to describe `H_FALSE` directly instead. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches (same simulation method as the last two turns).

- **Serial init code (`LDA #$03` through `STA ACIACR` , including the `SERIALPOLL` mode-select) extracted from `COLDSTRT` into a new subroutine, `INITSERIAL` , and moved into the ACIA Interrupt section (`SECTION 3`), right before the `IFEQ SERIALPOLL` that gates `INFILL` .** `COLDSTRT` now just does `JSR INITSERIAL` where the inline code used to sit. The extracted block keeps its internal `SERIALPOLL` conditional (`CR_RXON` vs `CR_POLL`) exactly as it was, just relocated and given an `RTS` . Small, bounded side effect worth naming rather than silently ignoring: this shifts roughly 10 bytes of real content from `INITCODE` to `BASECODE` (the subroutine itself, plus the shrink from removing the inline code and adding a 3-byte `JSR` in its place) - well within the existing, already-acknowledged estimation uncertainty for both regions, not something that changes the tracked overlap numbers meaningfully. Verified: `IFEQ` / `ELSE` / `ENDC` still balanced (3/3/3 - moving a conditional block doesn't add a new one), `INITSERIAL` defined exactly once and correctly resolved by its `JSR` , zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches (same simulation method as the last several turns).

- **Real bug fixed: `INTERPRET` called `WORD` without pushing a delimiter character first, unlike every other caller of `WORD` in this file.** Found while tracing a full character-by-character simulation of the ACIA receiving "ABC"+CR under polling mode. `WORD` 's calling convention (`PULU D / STB DELIM`) requires its caller to push a delimiter char first - confirmed by checking all 7 call sites: `HEADER` , `TOW` , `ISW` , `SQUOTE` , `CHARW` , and `LPAREN` all correctly push one (`LDD #32/#34/#')'` then `PSHU D`) immediately before `JSR WORD` . `INTERPRET` was the only one that didn't. Traced the actual consequence: `QUIT` invokes the interpreter via `CATCH` , which only pulls the xt off `U` and pushes nothing back (its bookkeeping goes on `S` , the return stack, not `U`) - so `WORD` 's stray `PULU D` reads from an empty `U` stack. `SP0` (`U` 's empty/reset value) is exactly `RSTACK` 's first byte, not unused margin, so `DELIM` ended up holding live return-stack content instead of a real delimiter - unpredictable, not necessarily space, meaning `WORD` couldn't reliably find

token boundaries at all. Confirmed this is not polling-specific, per instruction: `IRQH` (the interrupt-driven receive path) never touches `U` at all, so interrupt-driven mode had no incidental workaround either - same bug, both branches. Fixed by pushing `LDD #32 / PSHU D` right at the `ILOOP:` label before `JSR WORD`, matching every other call site's convention exactly rather than inventing a different approach - this single insertion covers every iteration of the loop, since every branch back to `ILOOP` (from `DOEXEC`, `CCALL`, `TRYNUM`'s success path) re-enters at this exact point. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches (same simulation method as recent turns).

- **CORRECTED the same turn this was written, by real assembler data: the "new VECTORS overlap" claimed below did not actually exist - it was computed from a 78-byte manual estimate that turned out to be wrong by 7 bytes.** A real assembler run reports `INITCODE`'s actual size as 71 bytes (`$47`), not 78. At `$FFA9`, 71 real bytes end at exactly `$FFEF` - one byte below `VECTORS`, zero gap, zero overlap. The `BASECODE` overlap described below (12 bytes against its nominal budget) is unaffected by this correction and remains open. Kept as history, not deleted - this was a real, reasoned finding at the time, just based on the best information available before a real assembler had actually been run against this specific figure. Original text: **`INITCODE` shifted up 7 bytes (`$FFA2` -> `$FFA9`) - reduces one overlap but introduces a different, new one.** Computed precisely before and after: against `BASECODE`'s nominal 8110-byte budget, the overlap shrinks from 19 bytes to 12 (`$FFA9` - `$FFB4`) - real progress, not a fix. But `INITCODE`'s real content (78 bytes, an established figure from `COLDSTRT` + `WARM` + `WARMMSG`, not an estimate) used to end at `$FFEF`, exactly one byte below `VECTORS`' start (`$FFF0`) - zero gap. At `$FFA9` it now ends at `$FFF6`, 7 bytes *into* `VECTORS`' own territory - a brand new overlap that didn't exist before this change, and arguably more certain than the `BASECODE` one, since it's based on solid content rather than a budget estimate. Worth noting: checked what happens using `BASECODE`'s *real* 7984-byte content instead of its nominal budget - the `INITCODE` / `BASECODE` overlap disappears entirely under that assumption, leaving only the new `VECTORS` overlap. Applied exactly as requested; neither overlap resolved here. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches.
- **The real, assembler-confirmed figure: `INITCODE` is 71 bytes (`$47`), told directly rather than derived from a manual count - the first genuine assembler-reported figure in this whole project, as opposed to every prior byte count in this file, which has been a manual estimate with an explicitly acknowledged margin of error.** Corrected the one place in `forth6809.asm` that cited the old, wrong 78-byte figure (`INITCODE`'s own `EQU` comment), and the checklist entries above. Left two older, already-properly-hedged entries alone rather than edit them - they already said "still an

estimate… not a real assembler's output" at the time, so nothing in them is actually wrong, just superseded. `BASECODE` 's real size (7984 bytes, from the instruction-by-instruction manual count) remains unconfirmed by an actual assembler run - this correction applies specifically to `INITCODE` , not the other regions' still-estimated figures.

- **`TRUE` and `FALSE` replaced with simple subroutines, per explicit request - but the requested values are inverted from the rest of the system's true/false convention, and this was applied exactly as asked, not corrected.** `TRUEBODY` is now `LDD #$0000 / PSHU D / RTS` ; `FALSEBODY` is now `LDD #$FFFF / PSHU D / RTS` - both simple, direct subroutines (matching the style of e.g. `BLW`), replacing the DODOES-trampoline CONSTANT-pattern implementation they used before (`TRUEVAL` / `FALSEVAL` , the value cells that pattern needed, removed along with it - confirmed unreferenced anywhere else first). Flagging this prominently rather than as a routine note: `TRUEV EQU $FFFF` / `FALSEV EQU $0000` , defined near `GLOBALS` , are what every comparison operator in this system (`=` , `<` , `>` , and the rest) actually pushes for a true/false result - that convention is unchanged. `TRUE` and `FALSE` , the two words, now push the opposite of what those operators mean by "true" and "false." Any code that does something like `TRUE IF ... THEN` would behave backwards from what every comparison-based conditional in the same program does, since `0` is false-for-branching on this CPU regardless of which named constant produced it. Verified: zero duplicate symbols and dictionary chain still 219 entries intact in both `SERIALPOLL` branches; confirmed `DODOES` and `ATSIGN` remain genuinely used elsewhere (by `CREATE` / `CONSTANT` / `VARIABLE` 's own runtime code) and were not left orphaned by this change.
- **`TRUE` / `FALSE` corrected to the standard convention: `TRUE` pushes `$FFFF` , `FALSE` pushes `$0000` - each a direct `LDD #value / PSHU D / RTS` , no indirection.** The code found at the start of this turn had these inverted (`TRUEBODY` loaded `$0000` , `FALSEBODY` loaded `$FFFF`), with a comment explicitly documenting that inversion as deliberate, from an earlier request not captured in this checklist - likely predating a context boundary, since no corresponding entry exists above to mark as superseded. That comment also correctly flagged the consequence at the time: `TRUE` / `FALSE` disagreed with `TRUEV` / `FALSEV` (`$FFFF` / `$0000`) and every comparison operator (`=` , `<` , `>` , and the rest), which all return `TRUEV` for true and `FALSEV` for false. This turn's request restores the standard values, resolving that inconsistency. `TRUEVAL` / `FALSEVAL` (the old DODOES-trampoline value cells) were already removed in the prior change - nothing further to clean up there. Verified: `H_TRUE` / `H_FALSE` 's `FDB TRUEBODY` / `FDB FALSEBODY` CFA fields unaffected (only the pushed values changed, not the labels), zero duplicate symbols, dictionary chain still 219 entries intact in both `SERIALPOLL` branches.
- **`FIND` confirmed genuinely missing a dictionary entry - added.** Checked first, not

assumed: searched for any `H_FIND` label or `FCC "FIND"` string anywhere in the file, found neither. Before adding a header, verified `FIND` 's actual implementation against the standard ANS stack effect (`c-addr -- c-addr 0 | xt 1 | xt -1`) rather than assuming it matched: traced the success path (`FMATCH` / `FPUSH`) - pushes the xt, then checks the header's bit 7 flag, pushing `1` if set or `-1` (via `FISNORM`) if clear, matching immediate-vs-normal exactly - and the failure path (`NOTFOUND`) - reconstructs the original `c-addr` (`SNAMEP-1`) and pushes it alongside `0` . Both match the standard exactly; no code changes needed, only the header. Also added four simple number words - `1` , `-1` , `2` , `-2` - same pattern as `TRUE` / `FALSE` : direct `LDD #value / PSHU D / RTS` , no indirection. Code labels `ONEBODY` / `MONEBODY` / `TWOBODY` / `MTWOBODY` , since `1` / `-1` / `2` / `-2` aren't valid 6809 assembler labels. All five new headers (`H_FIND` , `H_1` , `H_M1` , `H_2` , `H_M2`) chained after `H_QUIT` , with `H_M2` now the true chain head - `BASELATEST` updated accordingly (still referenced symbolically by `COLD` , so no code change needed there either). Updated the two comments that named `H_QUIT` as the head to reflect the new one. Verified: zero duplicate symbols, dictionary chain now 224 entries (was 219), correctly ordered and ending at `0` , in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **`BASECODE` shifted up 35 bytes (`$E007 -> $E02A`) - two effects, neither an improvement.** (1) Opens a new 35-byte gap between `BASEDICT` 's real end (`$E006` , 1992 bytes) and `BASECODE` 's new start - previously exactly contiguous, zero gap. Not itself broken (a gap isn't a collision), just unused address space where there wasn't any before. (2) Worsens the existing `BASECODE` / `INITCODE` overlap: `BASECODE` 's nominal size (8110 bytes) is unchanged, so its nominal end shifted up the same 35 bytes too, from `$FFB4` to `$FFD7` - against `INITCODE` (`$FFA9`), the overlap grows from 12 bytes to 47. Full pairwise sweep confirms this is the only overlap anywhere in the memory map, just larger than before. Applied exactly as requested; neither effect resolved here. Also fixed two separately-stale comments caught while making this change: the `ORG BASECODE` line still cited `$DFF4` (two shifts behind), and `BASEDICT` 's own comment still claimed `BASECODE` "moved again to match," which is no longer true now that a gap exists between them. Verified: zero duplicate symbols and dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.
- **Real bug, found via actual MAME debugger single-stepping (not static analysis) and fixed: `WORD` 's `EMPTY` branch pushes a c-addr (`WORDBUF` 's address, matching its normal contract) that `INTERPRET` never consumed when the parsed token was empty.** `INTERPRET` 's check after `JSR WORD` was `LDX ,U` (a peek, not a pop) `/ LDA ,X / BEQ IDONE` , and `IDONE` was a bare `RTS` - meaning on every line, once nothing remains to parse, the final `WORD` call's returned address gets stranded on `U` instead of being consumed the way the `JSR FIND` path does (via `FIND` 's own `PULU X`). This happens on

every line, including successful single-word ones - it's just invisible there since nothing inspects the stack afterward. On lines with more content, the stranded address sits underneath legitimate values and accumulates across lines with nothing ever cleaning it up. Diagnosed collaboratively: static tracing of `WORD` / `FIND` / `EXECUTE` / `DOT` 's entire numeric chain across several turns kept coming back clean (correctly - none of them had the bug), before an actual MAME debugger session found a spurious address sitting on top of an otherwise-correctly-pushed value, which pointed straight at this exact mechanism once checked against the code. Fixed by changing `IDONE` to `PULU X / RTS`, matching the same convention `FIND` already uses to consume a c-addr, rather than inventing a different fix. `IDONE` is referenced from exactly one place, so the fix at the label covers the only path that reaches it; confirmed `CATCH` (the caller once `INTERPRET` returns) doesn't rely on `X` at that point either. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real, serious bug found via MAME debugger and fixed: `WORD` stored the wrong length byte for every token followed by more input on the same line - `TFR X,D` (part of the `TOIN` calculation) silently destroyed `B`, which is `D` 's low byte and also where `SCANLP` 's own `INCB` loop had been counting the token's true character count.** `STB ,X+` then stored `TOIN` 's delta instead of that true count - one too many, since the delta includes the consumed trailing delimiter. The copy loop then copied one byte past the token's real end, corrupting every multi-token line's first N-1 tokens (the last token on a line is terminated by running out of buffer, not by `CONSUME`, so its `TOIN` delta happens to equal its true length - which is exactly why every single-token-line test earlier in this conversation, including my own manual traces, never surfaced this). This also means my own earlier traces of this exact routine, across several turns, were themselves quietly wrong in this specific respect - not because the routine's overall logic was mis-modeled, but because I treated `B` and `D` as if they could independently hold different values, when `B` is `D` 's low byte and any write to `D` clobbers it. Diagnosed collaboratively: found via actual MAME debugger single-stepping, not static tracing - confirmed against the source once precisely described. Fixed by saving `B` (`PSHS B`) before the `TFR X,D` / `SUBD SRCADDR` / `STD TOIN` sequence and restoring it (`PULS B`) immediately after, right before `STB ,X+` - placed at the `ENDW` label itself, which both paths that reach it (`CONSUME` 's fall-through and `SCANLP` 's direct branch when the buffer runs out) correctly pass through. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real, serious bug found via MAME debugger and fixed: `CCALL` (compiles a `JSR` to a given xt - used by every colon definition to compile a call to each word it contains) corrupted the target address on every single call.** `PULU D` pulled the xt first; `LDA`

`#OPJSR` (loading the opcode to compile) then overwrote `A` - which is `D`'s high byte, so this silently destroyed the top byte of the address just pulled. `STA ,X+` correctly wrote the opcode (A happened to hold the right value for that specific write), but `STD ,X++` then wrote the corrupted `D` out as the call target - a `JSR` to a garbage address, one wrong byte away from the real target. This affects every colon definition unconditionally, not an edge case - `: tv . ;` failed exactly this way, confirmed via MAME debugger as "attempted execution of random memory." Fixed by deferring `PULU D` until after `STA ,X+`: `LDX CODEHERE / LDA #OPJSR / STA ,X+ / PULU D / STD ,X++ / STX CODEHERE / RTS` - `D` is never live at the same time `A` gets reused for the opcode, so nothing clobbers it. The external calling convention (caller pushes the target address, then `JSR CCALL`) is unchanged - only the internal instruction order moved, confirmed by checking a caller still correctly pushes the address before the call. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed: `CONSTANT` compiled a self-referential PFA field instead of one pointing at the actual value cell.** `LDD CODEHERE` read `CODEHERE`'s value *before* the `JSR CODECOMMA` that writes the PFA field itself - at that point `CODEHERE` is the address of that very cell, not of the value 2 bytes further on where the subsequent `JSR COMMA` appends the real number. The compiled structure ended up as `[JSR DODOES][FDB ATSIGN][FDB <address of itself>][value]` - `DODOES` / `@` correctly fetches through the PFA, but the PFA pointed at itself, so executing the constant returned its own compile-time address instead of the stored value. `1234 CONSTANT c1` then executing `c1` returned the `CODEHERE` address, not `1234`, exactly matching the debugger trace. Checked `VALUEW` (the structurally similar routine right below `CONSTANT`) for the same issue and confirmed it's genuinely different, not just superficially similar - its PFA points into `VARHERE` (a separate mutable region), and the value is stored at that same `VARHERE` position via `VCOMMA`, so no self- reference exists there; left unchanged. Fixed `CONSTANT` by adding `ADDD #2` after `LDD CODEHERE`, accounting for the PFA field's own 2-byte width so it correctly lands on the value cell that follows rather than on itself. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed in three places: `DOTEST` (the `LOOP` runtime), `DOPLUSTEST` (the `+LOOP` runtime), and `LEAVE` all popped the return address off `S` (`PULS X`) before finishing their fixed-offset reads/writes of the loop-control cells `DOSETUP` had pushed - shifting every subsequent `2,S` / `4,S` / `6,S` access by 2 bytes.** Confirmed the exact layout `DOSETUP` pushes (`0,S` =return addr, `2,S` =index, `4,S` =limit, `6,S` =leave-flag) and that those specific offsets were correctly designed for that unshifted layout - the only bug was the premature pop happening

before they were read. `: lpy 10 3 DO I . LOOP ;` failed exactly this way: `DOTEST`'s `LDD 6,S` (meant to check the leave-flag) instead read straight through into whatever return address was already on `S` below the loop cells (`EXECUTE`'s own, in this case) - almost always nonzero, so `BNE DTEXIT` fired on the very first pass, exiting the loop immediately. Fixed all three by deferring `PULS X` until each path (continue vs. exit) actually needs it, rather than popping once up front - `DOTEST` / `DOPLUSTEST` each need it in two separate places (the normal-continue path and the exit/ `DTEXIT` / `DPTEXIT` path), so it's deferred separately in each rather than moved to one shared spot. Checked `?DO` for a separate copy of this bug and confirmed it reuses the same, now-fixed `DOTEST` via `LOOP`'s own compilation - no separate fix needed there. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real, systemic bug found via MAME debugger and fixed in seven places: `LOOP`, `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, and `ENDOF` all saved a branch target in `X` (`PULU X`), then called `CCALL` and/or `CODECOMMA` before using it - both of which internally reload `X` (`LDX CODEHERE` / `LDX #CODEHERE` via `APPENDCELL`), silently destroying the saved target before it was ever used.** Found while confirming a specific report on `LOOP`: `: lpy 10 3 DO I . LOOP ;` compiled a branch displacement of `+8` instead of `-8`, since `TFR X,D` read the address of the `CODEHERE` variable itself (left there by `CODECOMMA`'s internal `APPENDCELL`) instead of the real loop-body target `PATCH` needed - `DOTEST` then returned to a near-zero, invalid address. This had been masked until the `DOTEST` return-stack-offset bug (an earlier turn this session) was fixed - the loop never previously executed far enough to reach this code path. Once confirmed on `LOOP`, swept every other `PULU X` in the file rather than fix only the reported case: found the identical pattern in `+LOOP` (`DOPLUSTEST`'s compile side), `UNTIL`, `AGAIN`, `REPEAT`, `ELSE`, and `ENDOF` - seven total. Two different fixes depending on structure: where nothing else touched `U` between the pop and the eventual use (`LOOP`, `+LOOP`, `UNTIL`, `AGAIN`, `REPEAT`), parked the target on `U` itself (immune to `CCALL` / `CODECOMMA`, which only touch `D` / `X` / `A`) and retrieved it via a later `PULU D` in place of `TFR X,D`. Where something else (`CODEHERE`) got pushed onto `U` in between (`ELSE`, `ENDOF`), parking on `U` would have retrieved the wrong value - used the `MSCR` scratch variable instead. Caught this distinction the hard way: an initial attempt at the `ELSE` / `ENDOF` fix incorrectly reused the park-on-`U` approach and would have retrieved `CODEHERE` instead of the saved target; caught and corrected before delivery by re-tracing the exact `U` sequence rather than assuming the same fix pattern applied uniformly. Verified every other `PULU X` in the file individually and confirmed none of the remainder are vulnerable - either no `CCALL` / `CODECOMMA` call sits between the pop and the use (`ECPATCH`, `THEN`, the `?DO`-forward-patch tails in `LOOP` / `+LOOP`,

`ISFOUND` / `AOFOUND`), or the routine doesn't compile code at all (the plain runtime memory/stack words). Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed: `IWORD` (`I`) and `JWORD` (`J`) each bracketed a fixed-offset return-stack read with an unnecessary `PULS X` / `PSHS X` pair, shifting `S` by 2 before the read - so `I` returned the loop limit instead of the index, on every iteration.** `: lpy 10 3 DO I . LOOP ;` now runs the correct number of iterations (the `DOTEST` /register- clobbering fixes from earlier this session), but printed the limit (10) ten times instead of the index (3..9). Traced against the layout established while fixing `DOTEST` : `DOSETUP` pushes `[return-addr@0,S][index@2,S][limit@4,S][leave-flag@6,S]` , and `2,S` unshifted already correctly targets the index - the `PULS X` / `PSHS X` pair served no purpose (nothing needed offset 0 for anything in either routine) and only shifted every subsequent offset by 2, landing one field over. `JWORD` had the identical bug at its own offset (`10,S` , correct for the outer loop's index in a nested `DOSETUP` layout, but wrong once shifted). Fixed both by removing the pop/push pair entirely, per the suggested fix, leaving the existing offsets (`2,S` , `10,S`) unchanged since they were already correct for the unshifted stack. Swept every other `PULS X` in the file for the same bracketing pattern before concluding the fix was complete: checked `UNLOOP` and `EXITUNLOOP` specifically, since both are directly related to loop-frame teardown - confirmed both are structurally different and correct as-is, since they discard *counted blocks* of bytes (`LEAS 6,S` / `LEAS 8,S` in a loop) rather than reading a value at one fixed offset, so they're unaffected by any temporary shift regardless. Verified: zero duplicate symbols, dictionary chain still 224 entries intact in both `SERIALPOLL` branches; byte-exact split-file reassembly.

- **New capability, not a bug fix: an assembly-level unit test framework added, with its first test (`TSTDUP`) written.** Gated by a new conditional-assembly flag, `UNITTESTS` , matching this file's established `IFEQ` convention (`0` = included, `1` = excluded entirely). Two insertion points: a `JSR TSTRUNNER` call at the true end of `COLDSTRT` (after `INITSERIAL` , before `JMP COLD` - at that point `U` / `S` are already valid, since `COLDSTRT` sets them at its very start, but nothing else - `APPVARS` / `DPHERE` / `CODEHERE` / `LATEST` / `BASE` - has been initialized yet), and the test framework's own body, living in what was previously unused ROM space right after `INOUT` 's shadow (`USROMSTRT` , `$C100`) - pure `FILL` padding before this existed. `UNITTESTS=1` reverts this block to exactly that padding, computed automatically via the `ROM` label rather than a fixed byte count (`FILL $FF,BASEDICT-ROM` , was `BASEDICT- USROMSTRT`), so it stays correct either way without needing hand-adjustment when the test code's size changes. Each test is independent by construction, per the stated design principle: it saves the data stack pointer (`U`) before touching anything and unconditionally restores it at the end regardless of pass or

fail, so a failed assertion mid-test can never leave stack residue for the next test to inherit. Test scratch variables live at the very start of `APPVARS` - safe only because tests run strictly before `COLD` initializes `VARHERE` to that same address, which correctly and immediately re-purposes that space afterward; this is a real constraint worth remembering if the call site ever moves. Reporting is shared across every test via `TSTREPORT` rather than duplicated per test: prints the test's name (a counted string, via `COUNT` + `TYPE` - the same mechanism `BADWORD` itself uses to print a failing word), then " OK" or " FAIL", then a `CR` . `TSTDUP` verifies both the stack's contents after `DUP` (the duplicate and the original both equal a deliberately non- trivial pushed test value - not `0` , `1` , or `-1` , so a test that only appears to pass due to a special-cased value would be caught - and a sentinel pushed beneath is confirmed undisturbed) and the data stack pointer's movement (exactly one cell, 2 bytes - `DUP` 's own net effect, isolated from the two setup pushes by capturing `U` immediately before and after the `JSR DUP` specifically, not around the whole test). `TSTRUNNER` and `TSTSTACK` are structured as extensible dispatchers (`JSR` a list of test groups, `JSR` a list of tests within each group) so more tests and more groups can be added without restructuring what's here. Verified across the full 2x2 matrix of `SERIALPOLL` / `UNITTESTS` combinations: zero duplicate symbols, dictionary chain intact (224 entries) in all four; byte-exact split-file reassembly. Not yet run on real MAME hardware - this is a first try, per the request, with exactly one test written; the framework's own correctness (not just `DUP` 's) still needs confirming against actual execution.

- **Three real bugs found by inspection (not MAME) and fixed: `DEFER` , `2CONSTANT` , and `MARKER` all had the identical self-referential PFA bug already confirmed and fixed in `CONSTANT` via the MAME debugger.** Flagged by a parallel 68000 port of this codebase, which spotted the same construction pattern (`LDD CODEHERE / PSHU D / JSR CODECOMMA` , writing the PFA field's own current address into itself before the real value is appended two bytes later) recurring in these three defining words, unfixed. Verified each by inspection rather than deferred to hardware testing, since this is a deterministic compile-time address computation, not a timing/register- interaction issue - the same reasoning already used to confirm `VALUEW` was *not* affected when `CONSTANT` was first fixed. Traced both the compile-time construction and the runtime consumer for each: `DODEFER` 's `LDD ,X` would have jumped to the PFA's own address on execution of any `DEFER` 'd word (crash); `DOMARKER` 's four fixed-offset reads (`,X` / `2,X` / `4,X` / `6,X`) would have restored garbage `DPHERE` / `CODEHERE` / `VARHERE` / `LATEST` values - the most severe of the three, since using a `MARKER` -created word would corrupt live dictionary state, not just misbehave locally. Also checked, and confirmed genuinely unaffected rather than assumed safe by association: `DEFER@` / `DEFER!` (go through `TOBODY` , a runtime offset computation from an existing xt - structurally immune, never compiles a self- referential field), `IS` / `ACTION-OF` (construct no PFA at all, just locate a word and call

DEFERSTORE / DEFERFETCH), and BUFFER: (BUFFERCOLON , checked because it sits directly next to 2CONSTANT in the source - uses the VALUEW -style pattern, PFA into VARHERE rather than CODEHERE , and VALLOT only advances VARHERE without ever writing through the PFA, so no self-reference exists there either). Fixed all three with the same ADDD #2 pattern already established for CONSTANT . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four SERIALPOLL x UNITTESTS combinations; byte-exact split-file reassembly. Not yet confirmed via MAME - the mechanism is identical to the already- hardware-confirmed CONSTANT bug, but these three fixes themselves are still unverified against real execution.

- **Real bug found by inspection (not MAME) and fixed: 2@ (DFETCH) read the two cells in the opposite order 2! (DSTORE) actually writes them, so a 2! 2@ round trip swapped the values.** Flagged by the parallel 68000 port. Traced both routines precisely: DSTORE writes x1 to the low address and x2 to the high address; DFETCH was reading the high address first (pushing it deep) and the low address second (pushing it on top) - the reverse order. Confirmed via a concrete round-trip trace ([x1, x2, a-addr] before 2! came back as [x2, x1] after 2@), which is deterministic and fully verifiable from source - no MAME needed to establish it, same reasoning as the MARKER / 2CONSTANT / DEFER PFA bugs. 2! itself was already correct and internally consistent; only 2@ 's read order was wrong. Fixed by swapping DFETCH 's two LDD / PSHU pairs to read low-then-high instead of high-then-low. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four SERIALPOLL x UNITTESTS combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.

- **Simplification (not a bug fix): the PULS X / PSHS X pair left over in LEAVE from the earlier DOTEST -class fix was a genuine no-op and has been removed.** Flagged by the parallel 68000 port, which dropped it rather than port a verified no-op forward. Confirmed by inspection: nothing runs between the pop and the push (no call, no other touch of S or X), so X held exactly what it held before the pop and PSHS X restored S to exactly where it already was - RTS would find the same return address there either way. Unlike DOTEST 's own deferred PULS X (genuinely load-bearing - its value feeds LDD ,X / LEAX D,X afterward), LEAVE never uses X 's value for anything; the pair was vestige of the pre-fix structure (where PULS X ran first and PSHS X was needed to restore S afterward), not something the fix itself required. LEAVE is now just LDD #TRUEV / STD 6,S / RTS . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four SERIALPOLL x UNITTESTS combinations; byte-exact split-file reassembly.

- **Simplification (not a bug fix): U. (UDOT) and U.R (UDOTR) each had a literal pop-then-immediate-push-back on the value being formatted - a genuine no-op, removed.** Flagged by the parallel 68000 port, same class as the LEAVE finding

immediately above. Confirmed by inspection: in both routines, `PULU D` immediately followed by `PSHU D` with nothing in between leaves `U` exactly as it was: the following `LDD #0 / PSHU D` (building the double-number `NUMGT` needs) pushes `0` on top of the value either way, whether or not it was ever popped and pushed back first. In `UDOTR` specifically, only the *second* pop/push pair (the value) was the no-op - the first (`PULU D / STD DRWIDTH` , consuming the width argument) is genuinely functional and was left untouched. `UDOT` now starts directly with `LDD #0` ; `UDOTR` now goes straight from the width pop to `LDD #0` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found by inspection (not MAME) and fixed: `UNESCAPEW` double-decremented `UESRCLEN` for a lone trailing backslash, underflowing the counter so the loop would run past the end of the intended input.** Flagged by the parallel 68000 port. Traced precisely: on encountering `\` , the escape-handling path decrements `UESRCLEN` once (correctly, accounting for the backslash itself) and checks `BEQ` - if the backslash was the *last* character in the input, `UESRCLEN` is now correctly `0` , but the branch fell into the shared `UEPLAIN` (plain-character) path, which decrements `UESRCLEN` a second time for a character that doesn't exist. `0 - 1` underflows to `$FFFF` (nonzero), so the next `UELOOP` pass's `BEQ UEDONE` never fires and the loop reads memory past the intended input. Confirmed the normal paths are unaffected: an ordinary plain character decrements exactly once (`BNE UEPLAIN` from the top); a real two-character escape like `\n` decrements once for the backslash and once more via the shared `UEEMIT` path for the second character - correctly 2 total for 2 characters consumed. The bug is specific to the lone-trailing-backslash case, where the escape branch's own decrement already accounted for the backslash before falling into a path that decrements again. Fixed by giving that one case its own landing point, `UELASTBS` - emits the backslash literally (`A` still holds it from the earlier `LDA ,X+` , so nothing needs reloading) without re-decrementing `UESRCLEN` , since it's already correctly `0` . `UEPLAIN` itself, still used by the normal plain-character path, was left completely untouched. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Design change (not a bug fix): `UNLOOP` is now a true no-op, resolving the `UNLOOP` / `EXIT` return-stack corruption from the prior turn.** Reasoning: traced `EXIT` (via `EXITUNLOOP`) across every scenario - no enclosing `DO` (compiled count `0` , the unwind loop never executes, falls straight through to a plain return), one enclosing `DO` with no prior manual `UNLOOP` (discards the correct, still-intact 8-byte frame). Both correct. The only broken combination was `UNLOOP` immediately before `EXIT` : `UNLOOP` 's

own discard (6 of 8 bytes) left the frame partially gone, then `EXITUNLOOP` unconditionally discarded a full 8 more, overshooting into whatever sat beneath - typically the enclosing word's own caller's return address - branching into random memory on return, requiring a soft reboot to recover. Given `UNLOOP` has no legitimate use other than immediately preceding an exit from the definition, and `EXIT` already handles that correctly and automatically on its own, the conflict is resolved by removing the second, redundant discard mechanism rather than by making `EXIT` 's runtime detect how much of the frame remains (a more complex, riskier change). Checked for a new failure mode before applying: `UNLOOP` called without an immediate exit, then falling through to `LOOP` / `+LOOP` again - already broken before this change too (`DOTEST` / `DOPLUSTEST` already expect the frame to still be there), so this introduces no new risk, and actually removes one instance of it. `UNLOOP` is now just `RTS` . Documentation updated to match: `UNLOOP` 's glossary entry rewritten to describe the no-op and why, kept in the dictionary for source compatibility with code written for other Forth systems where `EXIT` doesn't auto-unwind; `EXIT` 's own entry given a cross-reference noting `UNLOOP` isn't required beforehand. The Assembler Source appendix (Section 8.13, Control Flow) was updated to match the real source rather than left stale. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly; both new documentation passages confirmed present in the rendered PDF. Not yet confirmed via MAME.

- **Real bug found by inspection (not MAME) and fixed: `CASE` pushed a `0` onto the compile-time `U` stack that nothing downstream ever consumed, causing every `CASE...ENDCASE` construct - even the simplest, with no `OF` clauses at all - to throw `-22` at the closing `;` .** Reported via `: Nname CASE 0 OF ENDOF ENDCASE ;` ; traced with the debugger to a `CSP` mismatch of exactly one stray cell. Verified the full compile- time lifecycle: `CASE` pushes `[0, TAGCASE]` ; `OF` pushes `[placeholder-addr, TAGOF]` ; `ENDOF` pops exactly those two and pushes its own `[NEWFLD, TAGENDOF]` ; `ENDCASE` loops popping `TAGENDOF` / `NEWFLD` pairs until it sees `TAGCASE` , then stops. None of `OF` / `ENDOF` / `ENDCASE` ever read or pop past `TAGCASE` - the `0 CASE` pushed beneath it is structurally unreachable by every consumer, leaving it permanently stranded one cell below where `CSP` (set by `:` before `CASE` ever runs) expects the depth to return to. `;` compares against `CSP` and correctly detects the mismatch every time. This implementation's `ENDCASE` uses a `TAGCASE` -scan approach entirely, with no counter involved anywhere in its actual logic - the stray `0` looks like a genuine leftover from an earlier, counter-based design that was never fully removed when the scan-based approach replaced it. Fixed by removing `CASEW` 's `LDD #0 / PSHU D` - now just `LDD #TAGCASE / PSHU D / RTS` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file

reassembly. Not yet confirmed via MAME.

- **Correction to an earlier fix, found via a real MAME test (`MULT-TABLE`) and confirmed by inspection: `JWORD` 's offset, "fixed" to `10,S` a few turns ago, was itself wrong.** `J` returned a fixed value (the outer loop's limit) on every iteration instead of the progressing index - `11 1 DO 11 1 DO I J * . LOOP CR LOOP` printed the same row ten times instead of a real multiplication table. The earlier fix incorrectly assumed `DOSETUP` 's own `JSR` -pushed return address persists on `S` after each nesting level - it doesn't, since `DOSETUP` 's own `RTS` pops and consumes it to jump into the loop body, so it never actually occupies a slot for `J` to count past. Re-traced by counting every real push and pop across a nested `DO` rather than assuming a fixed frame size per level: after the inner `DOSETUP` finishes, `S` is `[inner-index@0][inner-limit@2] [inner-leave@4][outer-index@6][outer-limit@8][outer-leave@10]` ; `J` 's own `JSR` pushes one more cell on top, landing outer-index at `8` . Offset `10` (the earlier fix) lands on outer-limit instead - a value that never changes across outer iterations, exactly matching the observed symptom. `IWORD` 's own offset (`2`) was re-checked against this same corrected reasoning and confirmed still correct - it only ever sits one frame deep, not two, so the earlier error (specific to counting across a second, nested `DOSETUP` call) doesn't apply there. `JWORD` is now `LDD 8,S` . This is being logged transparently as a correction to a prior fix, not presented as if it were right the first time - worth being honest that even a change I was confident in and verified structurally at the time turned out to need real hardware testing to actually confirm. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Real bug found via a real MAME test (`CREATE DOES>`) and fixed: `CREATE` had the same self-referential PFA-pointer bug already confirmed and fixed in `CONSTANT` , `DEFER` , `2CONSTANT` , and `MARKER` - but was never on the previously-flagged list, so it went unfixed until now.** Reported via `: ENUM CREATE , DOES> @ ;` - `DOES>` returned the address one cell before the value `,` actually stored, instead of the value's own address. Traced precisely: `CREATE` compiles `[JSR DODOES][FDB DOESRT0] [FDB <CODEHERE's current value>]` - the same `LDD CODEHERE` - without- `+2` pattern as the other four, so the PFA-pointer field pointed at itself instead of two bytes further on, where `,` appends the real value next. Confirmed the runtime consequence through `DODOES` itself: it reads the *value stored at* the PFA-pointer field and pushes that (by design - this is how `CONSTANT` 's indirection works correctly, `DODOES` following a pointer to the real data rather than holding the data directly). With the field self-referencing, `DODOES` pushed the field's own address instead of following through to where the real value lives - exactly "the address before the 16-bit constant" instead of "the address 1 cell further on," matching the report precisely. Checked `VARIABLE` (the structurally

similar routine immediately below `CREATE`, sharing the same `DOESRT0` trampoline setup) for the same bug rather than assume safety from resemblance alone - confirmed genuinely unaffected, since it uses the `VALUEW`-style pattern (`VARHERE`, a separate region) rather than `CODEHERE` self-reference. Also confirmed `SETDOES` needs no change - it patches a different field entirely (the behavior field, 2 bytes past `JSR DODOES`), fully independent of the PFA-pointer field this bug affects. Fixed with the same `ADDD #2` pattern already established for the other four. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **False alarm, resolved: a reported `2CONSTANT` / `2@` discrepancy (`666 777 2CONSTANT cc0` returning `666 $700C` instead of `666 777`; a separate `2@` test on a `2VARIABLE` appearing to read back in reverse order) turned out to be testing against a stale binary, not a live bug.** Traced the reported symptom carefully against the current source before this was resolved: `CREATE`'s `+2` fix (this session, correcting the same self-referential PFA-pointer class as `CONSTANT` / `DEFER` / `2CONSTANT` / `MARKER`) and `DFETCH`'s low-then-high read order (fixed to match `DSTORE` several turns earlier, flagged by the parallel 68000 port) both checked out correctly on paper against the reported symptom - neither reproduced the described behavior in static trace, which was the first signal something didn't add up. Confirmed only one `DFETCH` definition exists in the file (no stale duplicate). Resolved once the user rebuilt against the current `forth6809.asm`/split files and re-tested: the updated binary shows neither problem. Both are confirmed fixed by the prior `CREATE` and `DFETCH` changes already logged above - no additional code change was needed. Recorded here explicitly so the history is clear: this was a real, reasonable bug report at the time, not a mistake in reporting it - the fix had simply already landed in a turn the tested binary predated.

- **Real, significant bug found via MAME debugger and fixed: `QLOOP` unconditionally reset `STATE` to interpret mode at the start of every line, silently breaking any colon definition split across more than one line of input.** Reported via `: test0 TRUE IF ." Hy" ELSE ." Hee" THEN ;` compiling correctly on one line but failing with a `-22` (CSP mismatch) when split across several. `COLON` sets `STATE=-1` and `SEMI` sets it back to `0` (after checking `CSP`) - the correct, sole places `STATE` should change during normal operation. `QLOOP`'s own `LDD #0 / STD STATE`, run at the top of the per-line loop, was a third, redundant reset that fired every time `QUERY` read a new line - including lines in the middle of an still-open colon definition, regardless of whether `;` had actually been reached. `TRUE` (and everything after it on a continuation line) got interpreted and, for anything with a stack effect, executed instead of compiled - `TRUE` pushing `TRUEV` onto the data stack instead of being compiled, leaving a stray cell `CSP` correctly caught as a mismatch at `;`. Confirmed the fix's placement carefully rather than just deleting the

reset outright: `QUIT` is only re-entered on cold boot or an uncaught error routing back through `ABORT` - confirmed ordinary successful lines loop back to `QLOOP` directly, never `QUIT` - so moving the reset to run once at `QUIT` (rather than removing it entirely) still correctly forces interpret state exactly when ANS's own `QUIT` semantics call for it (including recovering from an error that aborts an unfinished colon definition, which deleting the reset outright would have left permanently stuck in compile mode), just not on every single line of an otherwise-uninterrupted session. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found via MAME testing and fixed, surfaced directly by the prior QLOOP/STATE fix: `QOK` skipped the `CR` echo together with the "ok" message whenever `STATE` was nonzero, silently dropping the line ending for every line of a multi-line colon definition after the first.** Once the QLOOP fix let multi-line colon definitions actually compile successfully, the echoed source no longer resembled what was typed - every continuation line ran together onto one visual line, since the terminal never saw the `CR` the user had genuinely pressed. Traced to `QOK: LDD STATE / BNE QLOOP` running *before* `JSR CRW` - while compiling, the branch away skipped both the `CR` and the "ok" message together, when only the "ok" message is actually supposed to be conditional (correctly not shown mid-definition). The error path just above `QOK` already got this right (unconditional `JSR CRW` before printing the error message) - only the success path had the bug. Fixed by moving `JSR CRW` to run unconditionally, first, with the `STATE` check (and the "ok" message it guards) coming after. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed: `FILL` corrupted its own remaining-byte count on the very first iteration, running far past the requested length.** Same register-clobber class as `CCALL` 's original bug: `FILLOOP` loaded `D` with the count, then `LDA FILLCHR` (needed for the fill byte) overwrote `A` - `D` 's high byte - before `SUBD #1` decremented what it believed was still the true count. The high byte took on the fill character's value instead, so the count almost never reached zero at the intended point. Fixed by keeping the count in `Y` instead of round-tripping it through `D` /memory each iteration - `Y` is untouched by loading the fill character into `A` , so the conflict is structurally impossible now, not just avoided by careful ordering. Also addressed the redundant scratch-register usage flagged alongside the bug report: the target address is now kept directly in `X` (via `TFR D,X`) rather than round-tripping through `FILLADDR` first, matching the same treatment given to the count. `FILLCNT` / `FILLADDR` (aliases for the shared `MVCNT` / `MVDST` scratch cells) became genuinely unused once the loop no longer touches memory for them - confirmed via search before removing, and removed

along with their comment-block entries, consistent with how `DICTTOP` was handled earlier this session. `MVCNT` / `MVDST` / `MVSRC` themselves are untouched, since `HSLEN` / `HSADDR` / `MRESULT` / `FILLCHR` still alias them for other routines. `ERASEW` (which `JMP` s into `FILLW`) confirmed unaffected, since only the internal loop changed, not the external calling convention. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real bug found via MAME debugger and fixed: `HEXBYTE` (used by `DUMP` 's hex column) read from `MSCR+1` instead of `MSCR` , formatting whatever stale byte happened to be at the never- written address instead of the byte actually passed in.** `STB MSCR` writes one byte, at `MSCR` itself; both subsequent reads (`LDB MSCR+1` , once for the high nibble via four `LSRB` s, once for the low nibble via `ANDB #$0F`) used the wrong address - a cell `HEXBYTE` never writes at all, so both nibbles were derived from stale scratch content rather than the real value. Reported via `MOVE + DUMP` : a fill character of `$7B` showed as `$03` in the hex column, while the ASCII column (a separate code path, unaffected) correctly showed `}` . Confirmed `MSCR` is shared, general-purpose scratch used pervasively elsewhere via full 16-bit `STD` / `LDD` - `HEXBYTE` 's single-byte `STB` / `LDB` pattern was the outlier, and nothing else depends on `MSCR+1` holding anything meaningful at the point `HEXBYTE` runs. Fixed by changing both `LDB MSCR+1` occurrences to `LDB MSCR` , reading from where the byte was actually stored. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Formatting change, not a bug fix: `DUMP` now emits a leading `CR` before its first line of output, matching the trailing `CR` `DULEND` already emits after every line (including the last).** Requested for consistent vertical alignment - without a leading `CR` , the first line started wherever the cursor already was (e.g. right after the echoed command itself), while every subsequent line correctly started at column 0 via the existing trailing `CR` . Added `JSR CRW` as the first thing `DUMPW` does, right after popping its two arguments and before the per-line loop begins. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.
- **Real bug found via MAME debugger and fixed: `CMOVE` never terminated - same register-clobber class as `FILL` 's bug earlier this session.** `CMVLOOP` loaded `D` with the count, then `LDA ,X+` (needed to fetch the byte being copied) clobbered `A` - `D` 's high byte - before `SUBD #1` decremented what it believed was still the true count, so the terminating condition almost never fired. Checked the two adjacent, structurally similar routines rather than assume they shared the bug: `CMOVEGT` (the reverse-direction,

overlap-safe variant) is genuinely unaffected, since its own copy (`LDA ,X`) happens *before* `LDD MVCNT` reloads the count fresh from memory each iteration, rather than after - the clobber happens, but the count is correctly reloaded afterward, overwriting it, not corrupted by it. `MOVEW` has no copy loop of its own at all - it's purely a dispatcher choosing `CMOVEW` or `CMOVEGT` based on overlap direction, so it's correct automatically once `CMOVEW` is. Fixed `CMOVEW` differently than `FILL` : unlike `FILL` (which only needed one address, leaving a register free for the count), `CMOVEW` already commits both `X` (source) and `Y` (destination) to the two addresses, leaving no spare 16-bit register to hold the count in in the same way. Fixed instead by reordering - decrement and store the count while `D` still holds the true value, before the byte copy is free to clobber `A` . Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly.

- **Real design gap found via MAME testing and fixed: `SUBSTITUTE` never returned the ANS-required substitution count, and its core algorithm was a plain substring search-and-replace on the bare registered name rather than the ANS `%` -delimited scan.** Confirmed against the actual spec (forth-standard.org/standard/string/SUBSTITUTE and …/REPLACES, both fetched and read in full): `SUBSTITUTE` must scan for text between `%` (ASCII `$25`) delimiter pairs specifically - `%%` collapses to a single `%` (count unchanged); a name matching the `REPLACES` registration has the *entire* `%name%` span, delimiters included, replaced by the substitution text (count incremented); a non-matching name is passed through unchanged, delimiters and all (count unchanged); a trailing `%` with no closing delimiter passes the residue through unchanged. Rewrote `SUBSTITUTEW` completely to implement this scan, no longer calling `SEARCHW` at all - confirmed `SEARCHW` still has its own dictionary header (`H_SEARCHW`) and remains a legitimate standalone word (`SEARCH`) independent of this change. Reused the existing `COMPAREW` for the name-matching step rather than writing a new comparison loop. `GLOBALS` is fully packed (256/256 bytes, confirmed) - the rewrite reuses `MSCR` / `MSCR2` / `MSCR3` / `MSCR4` (confirmed untouched by `SUBCOPY`) for the new algorithm's scan state rather than adding dedicated cells. Also fixed the missing third return value - `SUBSTITUTEW` now pushes the substitution count (`0` or positive on success) alongside the destination address and length, matching `( addr1 len1 addr2 len2 -- addr2 len3 n )` . The destination-overflow behavior (`THROW -1`) was left unchanged - the ANS spec explicitly permits throwing for negative- `n` cases in the standard THROW-code table.

```
**Separately identified, and deliberately NOT fixed on request:**
`S"`, when interpreted (not compiled - i.e. typed directly at
the prompt rather than inside a colon definition), always
writes into the same fixed, shared buffer (`SIBUF`), returning
that same address every time. `REPLACES` only stores the
```

```
addresses it's given, not copies of the text - so two `S"`
calls used to register a name/value pair (and a third `S"` for
`SUBSTITUTE`'s own source string) all silently overwrite the
same buffer, corrupting earlier registrations before
`SUBSTITUTE` ever runs. Traced precisely enough to reproduce a
reported garbled MAME output (`Dancinging, %girl%!`) by hand,
confirming the exact mechanism. User's explicit decision: don't
add dedicated copy-on-register storage to `REPLACES` - instead,
wrap each string in its own colon definition (`: name S"
text" ;`) for stable, independent storage, and document this
requirement in the glossary instead of changing the code. Done -
see the `REPLACES`/`SUBSTITUTE` glossary entries in
ClaudeForth.docx, now also corrected to match the real ANS stack
effects and behavior rather than the previous, inaccurate
descriptions.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly. Given the size
of the rewrite, long branches (`LBRA`/`LBHS`/`LBNE`) were used
liberally for loop-backs and larger jumps as a safety margin.
**MAME-CONFIRMED** (single and double substitution both
correct: `S" Dancing, %girl%!" poem 80 SUBSTITUTE` -> "Dancing,
Agnetha!"; `S" Two dancing girls- %girl%! %girl%!" poem 80
SUBSTITUTE` -> "Two dancing girls- Agnetha! Agnetha!", with
`.S` showing `2` as the substitution count) - real assembly and
execution confirm the long-branch margin was sufficient and the
`%`-delimiter algorithm itself is correct.
```

- **Verification task, requested and completed: confirmed PAD's current offset satisfies all three ANS transient-region minimums (forth-standard.org/standard/usage, section 3.3.3.6), added named equates documenting them, and redesigned S" to use PAD instead of a fixed, retired SIBUF.** Verification result: already satisfied, no numeric adjustment required. - PAD's own scratch region (>=84 chars): `PADW` computes `PAD = CODEHERE + 84` - meets the minimum exactly, with the usual ANS-expected transient/dynamic behavior (moves as CODEHERE advances). - WORD's transient region (>=33 chars): `WORDBUF` spans `$01DA` to `$01FA` (up to where the now-retired `SIBUF` began) - computed precisely via address subtraction (`$01FB - $01DA = 33`), meeting the minimum exactly. Unrelated to the PAD offset - this system's `WORD` uses its own separate, fixed buffer rather than HERE-relative storage. - Pictured numeric output buffer (>= (2*16)+2 = 34 chars): traced `LTNUM` (" <#") and `HOLD` precisely - `HLD` anchors to `PADW` 's own return value, and `HOLD` decrements *before* storing, so this buffer grows downward from PAD into the CODEHERE-to-PAD gap, not into PAD's own region above it. 34 <= 84, comfortably

satisfied with 50 bytes of margin.

```
Added `PADMINSIZE`/`WORDMINSIZE`/`HOLDMINSIZE`/`PADOFFSET` as
named, documented equates, replacing the bare `84` that used to
sit directly inside `PADW`.

Redesigned `S"`'s interpreted-mode path (`SQINTERP`) to write
into PAD (computed fresh via `PADW` on every call) instead of
the old, fixed `SIBUF` - giving interpreted `S"` access to
PAD's full 84-character region instead of the previous fixed
32-character limit. Confirmed `SIBUF` was used exclusively by
`S"` (matching the user's own stated assumption, verified by
search) before retiring its `EQU` definition; left its old
32-byte address range ($01FB-$021A) unclaimed rather than
shifting `APPVARS` down to reclaim it, to avoid any risk to the
rest of this carefully-verified memory map for the sake of 32
bytes on a system with headroom to spare.

**Difficulties check, as requested:** confirmed no conflict
with the pictured-numeric-output buffer (opposite growth
directions from the same PAD anchor - traced precisely, they
don't overlap). Confirmed a genuine, expected trade-off instead:
per ANS's own allowance ("non-standard words... may use PAD, but
such use shall be documented"), PAD is now shared between the
user's own general-purpose use and interpreted S" - using one
right after the other will overwrite one with the other, same
as any other transient-region interaction the standard itself
anticipates. Also identified and documented a subtler point: PAD
redesign does NOT eliminate the need for the user's own
colon-definition-wrapping workaround (established two turns
ago) - PAD's address only changes when something is compiled or
allocated between calls, so two interpreted S" calls in a row
(e.g. REPLACES's two arguments) still collide, just via PAD
instead of SIBUF. Compiled S" (inside a colon definition) never
touches PAD at all, writing directly into the definition's own
permanent storage instead - this remains the correct approach
for anything needing more than one string to coexist. Confirmed
`SIBUF` referenced nowhere else in the file before retiring it.

Glossary entries for `PAD`, `S"`, and `REPLACES` updated to
match - the `REPLACES` entry's prior `SIBUF`-specific wording
(from the previous SUBSTITUTE fix) was stale and has been
corrected to describe the current PAD-based mechanism instead.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
```

```
combinations; byte-exact split-file reassembly; both updated
Assembler Source appendix subsections (8.1 Memory Map and 8.20
String Words) and all three updated glossary entries confirmed
present in the rendered PDF. **MAME-CONFIRMED** as part of the
full substitution chain below - the PAD-based interpreted-mode
S" storage this entry introduced is exercised correctly by
every scenario in that confirmation.
```

- **Confirmed and fixed at the user's request: WORD had its own, separate, fixed 31-character-capped buffer (WORDBUF), entirely unrelated to CODEHERE or PAD - the PAD-based S" redesign two turns ago didn't help long S" strings because WORD truncates during parsing, an earlier stage than either S"'s storage path.** Redesigned WORD to scan directly into the CODEHERE-to- PAD gap instead, matching the traditional fig-Forth layout (WORD's buffer at the CODEHERE end, the pictured numeric output buffer at the PAD end, growing toward each other) - added `WORDMAXCHARS` as a named constant reserving `HOLDMINSIZE` bytes at the PAD end for that buffer, so the two don't collide even though ANS itself would permit the overlap (3.3.3.6). `WORDBUF` confirmed genuinely unused before retiring it, same treatment as `SIBUF` 's retirement two turns ago.

```
**Serious collision bug caught and fixed while verifying this
redesign, before delivery - not reported by the user, found by
tracing the change's own consequences.** `SQUOTE` (S"),
`DOTQUOTE` (."), and `AQSTOK` (ABORT") all compile a 3-byte
runtime trampoline (`JSR CCALL`) directly at `CODEHERE` *after*
calling `WORD` but *before* copying the parsed text - with
`WORD` now writing that same text at `CODEHERE` too, the
trampoline would overwrite the first bytes of the very text
being staged, before the copy loop ever ran. Checked every
other caller of `WORD` in the file systematically (`TO`, `IS`,
`ACTION-OF`, the main interpreter loop, `'`, `POSTPONE`,
`[']`, `CHAR`, `[CHAR]`, `(` ) - confirmed none of them share
this pattern, since they either resolve to an xt via `FIND`
immediately (never needing the raw text again) or extract a
single character into a register before any compilation runs.
`HEADER` (used by every defining word) is also unaffected,
since it writes into `DPHERE`, a completely separate region
from `CODEHERE`. Fixed all three affected words identically:
reserve 3 bytes ahead of `CODEHERE` before calling `WORD`, so
the parsed text naturally lands exactly where it needs to end
up, with the trampoline safely compiled into the reserved gap
in front of it rather than on top of it - the existing copy
loop becomes a harmless no-op (reading and writing the same
address) rather than needing to move anything.
```

Caught a further, related overflow while finishing this: the 3-byte reservation itself pushes the compiled path's text 3 bytes further into the CODEHERE-to-PAD gap than plain WORD-parsing does (e.g. a dictionary name for FIND) - the original `WORDMAXCHARS=49` would have let the worst case (S"/./ABORT", at the cap) overflow 3 bytes into the pictured-numeric buffer's reserved space. Corrected to `WORDMAXCHARS=46`, sized uniformly for the worst case across all callers rather than tracking a different effective limit per caller - simpler and safer.

Also caught and fixed a transcription error in my own edit before it was ever delivered: while inserting an explanatory comment into WORD's scan loop, the `BEQ ENDW` branch instruction immediately following the length-cap comparison was accidentally dropped, which would have made the length check compare but never actually branch - re-verified against the live file and corrected before proceeding further.

Glossary entries for `WORD`, `S"`, `."`, and `ABORT"` updated to match - `S"`'s entry from the previous turn specifically is corrected here, since its claimed "up to PADMINSIZE (84 characters)" limit was always inaccurate: WORD's own separate cap (31 at the time, now 46) was still the binding constraint, never 84.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-exact split-file reassembly; all four updated Assembler Source appendix subsections (8.1 Memory Map, 8.10 Outer Interpreter, 8.14 Compiling Words, 8.20 String Words) and all four updated glossary entries confirmed present in the rendered PDF.

MAME-CONFIRMED across every scenario that mattered, including the highest-risk one: `: expand S" Two dancing girls- %girl%! %girl%!" poem 80 SUBSTITUTE DROP SPACE TYPE ;` then `expand` -> "Two dancing girls- Agnetha! Agnetha!" - this specifically exercises the compiled-mode collision fix in `SQUOTE` (the reserve-3-bytes-before-calling-WORD pattern), confirming the runtime trampoline and the parsed text no longer overwrite each other. Also confirmed: the 34-character (and longer, 36-character) strings that originally exposed WORD's old 31-character cap now parse in full, in both interpreted (`S" ..." poem 80 SUBSTITUTE`) and colon-definition-wrapped (`: template S" ..." ;` then `template poem 80 SUBSTITUTE`) forms, with correct output in every case.

```
`DOTQUOTE` (`."`) separately MAME-confirmed too - its own
reserve-3-bytes fix, applied identically to SQUOTE's but as a
distinct edit, exercised independently: `: .message ." <text>
" ;` then `.message` types the text correctly, unmodified.

`AQSTOK` (`ABORT"`) also separately MAME-confirmed, completing
independent confirmation of all three fixed words: `: over18 18
< ABORT" Must be > 18" ;` - `3 over18` correctly types the
message and throws -2 (the true-flag path); `18 over18`
correctly falls through with no message and no throw (the
false-flag path). Both branches of ABORT"'s own conditional
logic exercised, not just the message-compilation mechanics
shared with SQUOTE/DOTQUOTE.
```

- **Real, significant bug found via MAME testing and fixed: `UNESCAPE` 's argument popping was completely wrong - only one of its three arguments was ever popped, and even that one went into the wrong variable.** ANS signature is `( c-addr1 u1 c-addr2 -- c-addr2 u2 )`. The code did `PULU D / STD UESRCLEN` (storing the popped `c-addr2`, the real destination address, into the *source-length* variable), then `LDD ,U` - a peek, not a pop - into *both* `UEADDR` and `UEDST` (misreading `u1`, the real source length, as an address, and never popping it off the stack at all). `c-addr1`, the actual source address, was never touched - left sitting on the stack the entire time. Net effect: `UEADDR` / `UEDST` both ended up holding a small length value misread as an address, the copy loop read and wrote through whatever garbage that pointed at (low memory, within GLOBALS itself, given typical string lengths), and the real source/dest addresses were either misfiled or left stranded on the stack - matching the reported symptom precisely (two addresses left on the stack, no real count, buffer untouched). Also fixed `UEDONE` 's return: it used to do `LDD UEOUTLEN / STD ,U`, overwriting whatever was left on top of the stack rather than pushing both required return values; now correctly pushes `c-addr2` (destination address) then `u2` (actual unescaped length), matching the ANS stack effect. Confirmed `UNESCAPEW` is referenced nowhere else in the file before applying the fix. The escape-processing loop itself (the `\n` / `\t` / `\\` / `\"` handling, including the lone-trailing-backslash fix from earlier this session) was untouched and unaffected - it operates correctly once its input variables are actually populated correctly. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Major finding, well beyond the prior turn's argument-order fix: `UNESCAPE` 's entire algorithm was wrong from the start, not just its argument popping.** After the previous fix corrected the broken argument order, MAME testing showed doubling a

single `%` wasn't happening at all, and the returned count didn't grow to account for it. Investigated against the actual spec rather than assuming a smaller bug within the existing logic - fetched both forth-standard.org/standard/string/UNESCAPE and the complang.tuwien.ac.at ANS Forth reference text, which includes the canonical reference implementation. Confirmed: `UNESCAPE` has nothing to do with backslash escape sequences (`\n` , `\t` , `\\` , `\"`) at all - the entire pre-existing implementation (predating this session) was decoding a plausible-sounding but entirely wrong operation. The real ANS `UNESCAPE` replaces every `%` character with two `%` characters and passes everything else through unchanged, specifically so that a literal `%` in text survives an eventual `SUBSTITUTE` pass unchanged ("If you pass a string through UNESCAPE and then SUBSTITUTE, you get the original string"). Replaced the entire routine body accordingly - confirmed the old backslash-handling labels (`UELASTBS` / `UEPLAIN` / `UECKT` / `UECKBS` / `UEEMIT`) were internal to this one routine before removing them. New output length is computed directly as the final write pointer minus the starting destination address, correct regardless of how many `%` characters were doubled, rather than incrementally tracked per-character the way the old (wrong) loop did it. Manually traced the new algorithm against the ANS reference's own test case (`"aaa%bbb" UNESCAPE` should equal `"aaa%%bbb"`) before delivery - confirmed matching exactly, including the length (7 characters in, 8 out). No destination-buffer-overflow check added, matching the ANS reference implementation, which also has none (an ambiguous condition per the standard, not a required error case). Glossary entry corrected completely - the prior entry's stack effect, description, and even its "result length is always <= input length" side-effect claim were all wrong, describing the old, incorrect algorithm rather than the real one (output can now be longer than input, by design). Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly; updated Assembler Source appendix subsection (8.20 String Words) and the corrected glossary entry both confirmed present in the rendered PDF. **MAME-CONFIRMED**: correctly escapes `%` characters as required.

- **Feature gap confirmed and addressed: the text interpreter had no support for ANS's double-number input convention (trailing '.') - "123." was silently treated identically to "123", leaving no way to get a genuine ud1 onto the stack for use with # and friends.** Confirmed against the precise spec text (forth-standard.org/standard/usage#usage:numbers, 3.4.1.3: "When the text interpreter processes a number... that is immediately followed by a decimal point... the text interpreter shall convert it to a double-cell number. For example, entering DECIMAL 1234 leaves the single-cell number 1234 on the stack, and entering DECIMAL 1234. leaves the double-cell number 1234 0 on the stack."). Traced `NUMBERQ` and found it already accumulated a full 32-bit value internally (`UDHI` : `UDLO` , via `NUMLOOP`) for every

number parsed, single or not - but only ever returned the low cell, discarding `UDHI` unconditionally, and had no detection of a trailing '.' at all. Also found the existing negation only negated the low 16 bits - harmless while only ever returning a single cell, but wrong once a genuine 32-bit value needed returning.

```
Redesigned `NUMBERQ`: detects a trailing '.' early (excluding
it from the digit count before `NUMLOOP` runs), performs full
32-bit two's-complement negation (low cell negated first, any
carry out of that addition propagated into the high cell,
manually traced against both a normal case and the zero-wraps-
to-zero edge case before delivery), and re-derives the double-
vs-single decision independently at the very end of the routine
by re-reading from `CADDR` and the original count - both
confirmed unchanged throughout the routine's entire execution,
including through `NUMLOOP`/`UDMULADD` - rather than remembering
the decision in a new persistent flag. This was a deliberate
choice: `GLOBALS` is fully packed at 256/256 bytes (the 6809's
own direct-page addressing limit, not an arbitrary number), so
avoiding a new flag byte avoided touching that boundary at all.
Return convention on success now distinguishes double (pushes
low, high, then success code `1`) from single (pushes value,
then the original `-1`, completely unchanged) - chosen so
`TRYNUM` doesn't need a separate flag either, and so single-
number behavior is provably identical to before by construction,
not just by testing.

Updated `TRYNUM` to handle both codes: on double success while
compiling, `UDHI` (on top of `U`) is stashed in `MSCR4` so `UDLO`
can be compiled as the first of two `LIT` literals, then `UDHI`
as the second - ensuring they execute in the order needed to
land correctly on the stack at runtime (low deep, high on top,
per 3.1.4.1's "the cell containing the most significant part...
shall be above the cell containing the least significant
part"). While interpreting, both cells are already correctly
placed by `NUMBERQ` and need no further handling.

Confirmed `TONUMBER` (`>NUMBER`) is untouched - it calls the
same shared `NUMLOOP`, but neither its own code nor `NUMLOOP`
itself needed any change; only `NUMBERQ`'s own wrapper logic
and `TRYNUM`'s consumption of it were modified.

Manually traced three cases by hand before delivery, beyond
structural verification alone: "123." (interpreted) -> [123, 0]
correctly, matching the ANS example exactly; "123" (no dot) ->
[123, -1] internally, correctly falling through to the original,
unchanged single-cell path, confirming backward compatibility;
```

```
"-123." -> UDHI=$FFFF/UDLO=$FF85, confirmed by hand to equal
$FFFFFF85 (i.e. -123 as a 32-bit value).

Glossary entries for `<#` and `#` updated with a note on how to
get a proper ud1 onto the stack from typed input, since that was
the specific gap originally reported.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly; updated
Assembler Source appendix subsection (8.10 Outer Interpreter)
and both updated glossary entries confirmed present in the
rendered PDF. Not yet confirmed via MAME - this is a
substantial, multi-path change (single/double x interpret/
compile x positive/negative all interact) and deserves thorough
real-hardware testing across that full matrix before being
considered settled.
```

- **Two real bugs found via a very thorough MAME test matrix (interpret and compile mode, both signs, single and double, cumulative `.s` traces across many lines) and fixed - both in the 32-bit negation logic delivered last turn for double-number input, and both only visible on negative doubles specifically.** Positive cases (`123. -> 0 123` , `123456. -> 1 -7616`) were already confirmed correct by the trace, matching the design exactly - the negation path (which only positive numbers skip entirely) was where the actual problems were.

```
**Bug 1**: the 6809's `COM` instruction (one's complement)
unconditionally sets the carry flag to 1, regardless of its
operand - a documented CPU quirk. The negation code computed
the real carry from the low cell's `ADDD #1`, but then ran
`COMA`/`COMB` on the high cell before checking that carry - and
`COMB` silently overwrote it with its own, always-set value, so
the intended "only propagate carry into the high cell when the
low cell actually overflowed" check never worked. `-123.`
returned high cell `0` instead of `-1`; `-123456.` returned `-1`
instead of `-2` - both exactly matching "always add 1"
regardless of the true carry, not the intended conditional
behavior. Fixed by saving the true carry via `PSHS CC`
immediately after the low-cell addition, before the high-cell
`COM` can destroy it, then `PULS CC` to restore it right before
the branch that depends on it.

**Bug 2, only exposed by fixing Bug 1**: with the carry check
now actually working, the branch it guarded (`BCC`) turned out
to jump all the way past `STD UDHI` itself when there's no
```

carry - not just past the `+1`. The complemented high-cell value was computed in D but never actually stored back to UDHI in that case, leaving it at its stale, pre-negation value. This was completely masked by Bug 1 in practice - since carry was always corrupted to "set," the branch was never actually taken, so the store always ran (with the wrong value, per Bug 1, but it did run). Fixing Bug 1 alone would have made this second, previously-latent bug live and produced a different set of wrong answers. Restructured so the store always runs, with the conditional `+1` applied before it rather than gating whether it happens at all.

Manually re-traced both negative-double test cases against the fully corrected code before delivery: `-123.` -> carry clear on the low cell, branch skips only the `+1`, raw complement `$FFFF` gets stored -> high cell `-1`, correct. `-123456.` -> same pattern, raw complement `$FFFE` stored -> high cell `-2`, correct (matches the true 32-bit value `$FFFE1DC0` computed by hand). Also re-traced the carry-set path (negating 0, as a sanity check) to confirm that branch still correctly falls through to the `+1` before storing.

Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL`x`UNITTESTS` combinations; byte-exact split-file reassembly; updated Assembler Source appendix subsection (8.10 Outer Interpreter) confirmed present in the rendered PDF. Glossary entries (`<#`, `#`) needed no further changes - this fix doesn't alter the stack effect or usage, only the underlying correctness.

MAME-CONFIRMED, both interpret and compile mode, across the full t0-t7 matrix: `-123.` -> high `-1` / low `-123`; `-123456.` -> high `-2` / low `7616` - both matching the hand-traced prediction from the prior turn exactly (high `$FFFF`/`$FFFE` respectively, confirmed independently against `$FFFFFF85` and `$FFFE1DC0`). Positive cases (`123.` -> `0 123`, `123456.` -> `1 -7616`) and single-cell cases (unaffected by either bug, per the negation code being skipped entirely) also confirmed correct. Both the carry-corruption bug and the masked missing-store bug are now verified fixed on real hardware, not just by static hand-tracing.

- **Two explicit, requested adjustments applied: BASECODE and BASEDICT origins shifted down $40 (64 bytes) each, to make room for growth since they were last positioned; UNITTESTS set to 1 (excluded) since unit tests don’t work yet and resolution has been postponed.** BASECODE : $E02A -> $DFEA . BASEDICT : $D83F ->

`$D7FF`. Applied exactly as requested, without independently recomputing the resulting gaps/overlaps against `INITCODE` and each other - this region's comments have historically tracked those in precise byte counts, but doing so accurately requires real assembled sizes (`BASECODESIZE` / `BASEDICTSIZE`, computed via `*-start` at each section's end), which structural verification here cannot determine - it checks label integrity and dictionary-chain structure, not location-counter arithmetic. The comments at both `EQU`s now say this explicitly rather than presenting a static estimate as if it were a real assembled count - confirm the resulting gaps/overlaps on real assembly.

```
Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations (checked with both `UNITTESTS` values regardless
of the file's own new default); byte-exact split-file
reassembly.
```

- **Real bug found via MAME debugger and fixed: `S>D` performed no sign extension at all - the same register-flag class of bug already documented once this session in `TRYNUM`.** `PULU` does not affect condition codes on genuine 6809 - `STOD` did `PULU D` then immediately `BPL SDPOS`, testing whatever flags an unrelated, earlier instruction happened to leave set, not the sign of the value just popped. `-123 S>D` returned high cell `0` instead of `-1`. Fixed by adding an explicit `TSTA` (testing D's high byte, whose bit 7 reflects the full 16-bit value's sign) immediately before the branch that depends on it. Confirmed `STOD` is referenced only via its own dictionary header before applying the fix - nothing else calls it directly. Checked the neighboring routines (`DTOS`, `DMAXW`) for the same pattern rather than assume they're clean by proximity alone - both confirmed unaffected: `DTOS` has no branch at all, and `DMAXW` already uses an explicit `CMPD` before its own branch. Verified: zero duplicate symbols, dictionary chain still 224 entries intact across all four `SERIALPOLL` x `UNITTESTS` combinations; byte-exact split-file reassembly. Not yet confirmed via MAME.
- **Real bug found via MAME testing and fixed: `SIGN` had the same `PULU`-doesn't-set-flags defect already found in `STOD` - plus a separate, genuine usage-pattern issue in the reported test definition, diagnosed but not a code bug.** Reported via `: format DUP ABS S>D <# #S SIGN #> TYPE ;` never printing a minus sign for either sign of input.

```
**Code bug**: `SIGN` did `PULU D` then immediately `BPL
SIGNDONE` - `PULU` doesn't affect condition codes on genuine
6809, so the branch tested whatever flags an unrelated, earlier
instruction happened to leave set, not the sign of the value
just popped. Fixed with the same `TSTA` pattern used for `STOD`.
Checked all four existing internal callers (`DOT`/`DOTR`/
```

```
`DDOT`/`DDOTR`, i.e. `.`/`.R`/`D.`/`D.R`) before concluding
anything about their own correctness - all four turned out to
be accidentally unaffected, since each runs `LDD SAVEN` (a
flag-setting load of the true signed value) immediately before
`PSHU D`/`JSR SIGN`, and `PSHU` doesn't disturb flags. This was
fragile, caller-side luck rather than `SIGN` being correct on
its own - confirmed precisely by the fact that a direct,
standalone use (the reported test word) had no such protection
and failed outright.

**Separate, non-code issue identified in the reported
definition itself**: even with `SIGN` fixed, `DUP ABS S>D <#
#S SIGN #>` would still never add a minus sign, traced
precisely - `#S` fully consumes its `ud` down to `0 0`, so
whatever sits on top of the stack immediately after `#S` is
always `0`, genuinely not negative, regardless of the original
number's sign. The true signed value (left underneath by `DUP
ABS`) never reaches the top of the stack in that sequence at
all. Standard idiom needs the signed value stashed on the
return stack and restored right before `SIGN`: `DUP >R ABS
S>D <# #S R> SIGN #>`. Not a defect - flagged so the fix to
`SIGN` itself isn't mistaken for resolving this specific
definition's own behavior too.

Verified: zero duplicate symbols, dictionary chain still 224
entries intact across all four `SERIALPOLL`x`UNITTESTS`
combinations; byte-exact split-file reassembly. Not yet
confirmed via MAME.
```

Structural duplication (identified, some resolved, some not)

- `:` / `CREATE` / `VARIABLE` 's header-building — resolved via `HEADER` (section 7).
- `COMMA` / `CODECOMMA` / `CCOMMA` / `VCOMMA` / `VCCOMMA` / `CCOMMA1` — resolved via `APPENDCELL` / `APPENDBYTE` (section 6).
- `<#` / `#>` recomputing PAD's address inline — resolved, both now call `PADW` (section 20 / section 6).
- No further known duplication, but the codebase was never given a full pass specifically hunting for more.

Real, load-bearing gaps

- **Software (XON/XOFF) handshaking is still not implemented.** Now that hardware

RTS/CTS handshaking is in place, this is the one remaining flow-control gap: this system still never transmits XON/XOFF and would not recognize either byte as a control signal if the remote device sent them — an incoming `$11` / `$13` is just queued as an ordinary character. Only relevant for links with no RTS/CTS wiring (plain 3-wire serial); would need a byte- interception layer between the input ring and `KEY` 's caller.

- **No hard boundary checks** anywhere `DPHERE` / `CODEHERE` / `VARHERE` grow toward each other or toward the stacks. `UNUSED` and `VUNUSED` both correctly *report* the distance to their boundary now (`CODETOP` and the newly-added `APPVARSEND` respectively), but nothing enforces either — a runaway compile can silently corrupt an adjacent region.

- **`VUNUSEDW` (the `VUNUSED` word) computed a meaningless number, not "how much `APPVARS` space remains" - a real, distinct bug, not just the absence of a check. Fixed.** `LDD #APPCODE / SUBD VARHERE` computed `APPCODE - VARHERE` (roughly `$7000` minus wherever `VARHERE` currently sits within `APPVARS`), which had nothing to do with `APPVARS` 's own boundary - looked like a copy-paste slip from `UNUSEDW` immediately above it (`LDD #CODETOP / SUBD CODEHERE` , correct for `CODEHERE` 's own boundary). Surfaced directly by a question about how `APPVARS` 's size is actually set: it wasn't - `APPVARS EQU $021B` defined only its start; there was no end/size constant anywhere. Fixed by adding `APPVARSEND EQU APPVARS+8000` (an exclusive upper bound, `$215B` , matching `CODETOP` 's own convention for `APPCODE`) and changing `VUNUSEDW` to `LDD #APPVARSEND / SUBD VARHERE` . Verified: at cold start (`VARHERE` = `APPVARS`), this computes exactly 8000, matching `APPVARS` 's documented size precisely; zero duplicate symbols; dictionary chain unaffected. `APPVARSEND` initially fell within `APPDICT` 's current range - expected at the time, the same, already-tracked `APPDICT` / `APPVARS` overlap, not something this fix caused. Since resolved by redefining `APPVARSEND` as `APPDICT-1` - see above.

- **`ENVTABLE` is incomplete.** Missing entries: `/HOLD` , `/PAD` (this build never fixed a capacity for either — filling them in would mean deciding a real bound first, not just picking a number), `MAX-D` , `MAX-UD` (need the dispatcher extended to push *two* cells for double-cell answers — current `ENVQUERY` only handles one), `WORDLISTS` / `FLOORED` (need the dispatcher to be able to report a recognized-but-false answer, distinct from "unrecognized name" — no such path exists yet).

- **`CATCH` -wrapped `QUIT` / `INTERPRET` rollback is scoped to one input line only.** A colon definition spanning multiple lines that fails partway through a later line only rolls back that line's contribution, not the whole definition back to `:` . A complete fix needs the region-pointer snapshot taken once at `:` and held until `;` or an error, not refreshed every `QLOOP` pass.

- **`ABORTHDR` 's `LINK` field is a placeholder** `0` in the consolidated/split source — must be set to the real prior `LATEST` value once final ROM layout/assembly order is fixed.

Never resolved after being explicitly raised

- **`J` doesn't generalize past one level of loop nesting.** No `J2` /deeper equivalent exists. A triple-nested loop's innermost body has no built-in way to reach the outermost index without manually replicating `J` 's offset arithmetic one frame further.
- **`LEAVE` 's correctness depends on always being textually inside the loop it affects** — never verified against a `LEAVE` called from a separately-defined word invoked from within a loop (which would read/set the wrong stack frame, or crash).
- **`WORD` 's 31-character cap is now inconsistent within the system.** `PARSE` / `PARSE-NAME` were deliberately rewritten to not share it, but `WORD` itself — and everything still built on it (`CHAR` , `[CHAR]` , header names, `FIND`) — still has it.
- **The optional Search-Order word set is not implemented.** Every word resolves through one unified dictionary chain rooted at `LATEST` ; there is no multi-wordlist support (`WORDLIST` , `GET-ORDER` / `SET-ORDER` , `ALSO` / `ONLY` , etc.). This was a foundational decision fixed since `COLDSTRT` 's first version, not an oversight — now documented explicitly (ClaudeForth documentation, Section 4.5) along with what adding it later would actually require: restructuring `FIND` into a loop over an active search order, changing `HEADER` to link into a selectable "current" wordlist instead of unconditionally into `LATEST` , and fixing the few words (`RECURSE` , `;` 's unsmudge step, `WORDS`) that read `LATEST` directly today.

Open design questions (not defects — deliberate unresolved choices)

- Whether `ALLOT` / `VALLOT` / `PICK` / `ROLL` / `BASE` should range-check their inputs against actual stack/region bounds. Currently none of them do, consistently, by choice rather than oversight.
- Whether any additional distinction like `TIB` / `SOURCE` (fixed terminal buffer vs. current input source) is needed anywhere else in the system where `EVALUATE` 's redirection might still cause a mismatch.
- `SWIH` 's throw code (`-99` in the consolidated source) was never assigned a real, deliberate value — it's a placeholder for "some hardware trap occurred," not a considered ANS-style code.

- `REPLACES` / `SUBSTITUTE` remain single-slot (one registered name/value pair, overwritten by the next `REPLACES` call) rather than a true multi-entry table. A real table needs its own storage layout and lookup structure — a deliberate scope decision made when `SUBSTITUTE` was completed, not an oversight.

What's solid (for reference, not action)

Stack manipulation (Core + Core Ext + return-stack transfer), all arithmetic (single + double + mixed precision), logic, comparison (single + double), full control flow including `CASE`, all defining words including `DEFER` / `MARKER` / `VALUE` / `TO` (`VALUE` now correctly targeting mutable space), memory and string operations (including a completed `SUBSTITUTE`), `SOURCE` / `EVALUATE` / `REFILL` mechanics, and the Tools word set (`.S` / `WORDS` / `DUMP`, with `DUMP`'s edge-case bug fixed) are complete and were traced/verified against concrete cases during the build, not merely asserted correct.